



Advance Javascript

▼ Execution Context & JavaScript Engine

JavaScript Execution Context & JavaScript Engine

Before learning Promises, Async/Await, Closures, and Hoisting, students must understand **how JavaScript actually executes code behind the scenes.**

What Happens When We Run JavaScript Code?

Consider the code:

```
var name = "Anand";

function greet() {
  console.log("Hello");
}

greet();
```

When JavaScript sees this code, it does **NOT** execute line by line immediately.

Instead, JavaScript performs two major steps:

1. Memory Creation Phase
2. Code Execution Phase

These two phases together are called an **Execution Context**

What is Execution Context?

Execution Context is the environment in which JavaScript code is executed.

Think of it as:

```
Execution Context =  
{  
  Memory Component  
  +  
  Code Component  
}
```

Memory Component

Stores:

```
var name = "Anand";
```

```
name -> "Anand"
```

and functions

```
function greet() {}
```

```
greet -> Entire Function
```

Code Component

Executes code line by line.

```
console.log(name);  
greet();
```

Real Life Example

Imagine a classroom.

Before class starts:

```
Teacher prepares attendance sheet  
Teacher prepares notes  
Teacher prepares assignments
```

This is:

```
Memory Creation Phase
```

Then:

```
Teacher starts teaching
```

This is:

```
Code Execution Phase
```

Same happens in JavaScript.

Types of Execution Context

JavaScript has mainly:

1. Global Execution Context (GEC)

Created when program starts.

```
console.log("Start");
```

As soon as file runs:

```
Global Execution Context Created
```

Only one GEC exists.

2. Function Execution Context (FEC)

Created whenever a function is called.

```
function greet(){  
  console.log("Hello");  
}  
  
greet();
```

When:

```
greet();
```

runs,

JavaScript creates:

Function Execution Context

for greet().

Visual Representation

Global Execution Context

```
-----  
| Variables      |  
| Functions      |  
-----
```

|
|
v

greet()

```
-----  
| Function Context |  
-----
```

JavaScript Engine

JavaScript Engine is software that understands and executes JavaScript code.

Examples:

Browser	Engine
Chrome	V8
Edge	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

What Does JavaScript Engine Do?

```
console.log("Hello");
```

Engine converts code into machine instructions.

Process:

```
JavaScript Code
  ↓
Parser
  ↓
AST
  ↓
Interpreter
  ↓
Compiler
  ↓
Machine Code
  ↓
Output
```

Components of JavaScript Engine

1. Parser

Checks syntax.

Example:

```
let a = 10;
```

Valid.

Example:

```
let = 10;
```

Invalid.

Parser throws error.

2. AST (Abstract Syntax Tree)

Parser converts code into a tree.

Example:

```
let x = 10 + 20;
```

AST looks conceptually like:

```
  =  
 / \  
x  +  
  / \  
10 20
```

3. Interpreter

Reads code quickly.

```
console.log("Hi");
```

Runs immediately.

4. Compiler

Optimizes code for performance.

Modern V8 uses:

Interpreter + Compiler

How JavaScript Executes Code?

Example:

```
var a = 10;

function square(num){
  return num * num;
}

var result = square(5);

console.log(result);
```

Step 1: Global Execution Context Created

Memory Phase

Memory:

```
a -> undefined
square -> function
result -> undefined
```

Diagram:

Memory

```
a      : undefined
square : function
result : undefined
```

Step 2: Execution Phase

Line 1

```
var a = 10;
```

Memory becomes:

```
a -> 10
```

Line 2

Function already stored.

No execution.

Line 3

```
var result = square(5);
```

Function called.

New Function Execution Context created.

Function Execution Context

```
square(5)
```

Memory:

```
num -> 5
```

Execution:

```
return 5 * 5;
```

returns:

25

Function Context removed.

Back to Global Context

result -> 25

Output:

```
console.log(result);
```

25

Call Stack

JavaScript manages contexts using a Call Stack.

Example:

```
function one(){
    two();
}

function two(){
    three();
}

function three(){
    console.log("Done");
}

one();
```

Execution:

Call Stack

```
| one() |  
-----
```

```
| two() |  
| one() |  
-----
```

```
| three() |  
| two()   |  
| one()   |  
-----
```

After completion:

```
Removed  
Removed  
Removed
```

Stack becomes empty.

Call Stack Rule

LIFO

```
Last In  
First Out
```

Example:

```
Push one()  
Push two()  
Push three()
```

```
Pop three()  
Pop two()  
Pop one()
```

Complete Flow

```
JavaScript File
  ↓
Global Execution Context
  ↓
Memory Creation Phase
  ↓
Execution Phase
  ↓
Function Call
  ↓
Function Execution Context
  ↓
Call Stack
  ↓
Return Value
  ↓
Function Context Removed
  ↓
Program Ends
```

▼ Hoisting

Hoisting is one of the most important JavaScript concepts and one of the most frequently asked interview topics. To understand hoisting properly, students must first understand **Execution Context** because hoisting happens during the **Memory Creation Phase**.

Real Life Analogy

Imagine a teacher enters a classroom and prepares everything before the lecture starts.

1. Attendance Sheet
2. Marker
3. Notes
4. Projector

Only after preparation does the teacher start teaching.

Similarly, before JavaScript executes code, it first scans the entire program and allocates memory for variables and functions.

This preparation phase is called:

Memory Creation Phase

The behavior that makes variables and functions available before their declaration is called:

Hoisting

What is Hoisting?

Definition

Hoisting is JavaScript's behavior of moving declarations to memory before code execution.

Important:

Declarations are hoisted
Assignments are NOT hoisted

Example 1

```
console.log(a);  
  
var a = 10;
```

Output:

undefined

Many students expect:

ReferenceError

But JavaScript does:

Memory Creation Phase

```
a → undefined
```

Execution Phase

```
console.log(a);
```

Output:

```
undefined
```

Then:

```
a = 10;
```

Memory becomes:

```
a → 10
```

Behind the Scenes

JavaScript internally behaves like:

```
var a;  
  
console.log(a);  
  
a = 10;
```

This is not actual code transformation but helps understand hoisting.

Visual Representation

Memory Phase

a -> undefined



Execution Phase

console.log(a)



undefined



a = 10



a -> 10

Example 2

```
console.log(name);  
  
var name = "Anand";
```

Output:

undefined

Why?

Memory phase:

```
name -> undefined
```

Execution:

```
console.log(name);
```

prints:

```
undefined
```

then

```
name = "Anand";
```

Function Hoisting

Functions are hoisted completely.

Example:

```
greet();  
  
function greet() {  
    console.log("Hello");  
}
```

Output:

```
Hello
```

Works perfectly.

Memory Phase

```
greet -> Entire Function
```

Unlike variables:

```
a -> undefined
```

Functions store the entire function body.

Visualization

```
Memory  
  
greet -> function greet(){  
    console.log("Hello");  
}
```

Therefore:

```
greet();
```

works before declaration.

Example

```
sayHi();  
  
function sayHi() {  
    console.log("Hi");  
}
```

Output:

```
Hi
```

Variable Hoisting vs Function Hoisting

Variable


```
console.log(a);  
  
var a = 10;
```

Output:

```
undefined
```

Function

```
show();  
  
function show() {  
    console.log("Hello");  
}
```

Output:

```
Hello
```

Interview Question

Why?

Because:

```
Variable:  
a -> undefined  
  
Function:  
show -> complete function
```

Hoisting with Function Expression

Students often get confused here.

Example:

```
show();

var show = function() {
  console.log("Hello");
};
```

Output:

```
TypeError
```

Memory Phase

```
show -> undefined
```

Execution

```
show();
```

JavaScript tries:

```
undefined();
```

Impossible.

Output:

```
TypeError: show is not a function
```

Visualization

Memory

show -> undefined

↓

```
show();
```

↓

```
undefined();
```

↓

TypeError

let and const Hoisting

Many students think:

```
let and const are not hoisted
```

This is WRONG.

They are hoisted.

But they behave differently.

Example

```
console.log(age);
```

```
let age = 25;
```

Output:

```
ReferenceError
```

Students ask:

"If hoisted, why error?"

Because:

```
Hoisted YES  
Accessible NO
```

Memory Phase

```
age -> allocated memory
```

But:

```
Not initialized
```

JavaScript puts it into a special area called:

```
Temporal Dead Zone (TDZ)
```

What is Temporal Dead Zone?

TDZ is the period between:

```
Variable Creation  
and  
Variable Initialization
```

Example

```
console.log(age);  
  
let age = 25;
```

Timeline:

```
Memory Creation  
  ↓  
age created  
  ↓  
TDZ  
  ↓  
console.log(age)  
  ↓  
ReferenceError  
  ↓  
age = 25  
  ↓  
TDZ Ends
```

Visualization

```
Memory  
  
age -> TDZ
```

↓

```
console.log(age);
```

↓

```
ReferenceError
```

const and Hoisting

Example

```
console.log(PI);  
  
const PI = 3.14;
```

Output:

ReferenceError

Same reason:

Temporal Dead Zone

Comparing var, let and const

Feature	var	let	const
Hoisted	✓	✓	✓
Initial Value	undefined	TDZ	TDZ
Accessible Before Declaration	✓	✗	✗
Reassignment	✓	✓	✗

Most Important Interview Example

Predict Output

```
var a = 10;  
  
function test() {  
    console.log(a);  
  
    var a = 20;  
}
```

```
test();
```

Output:

```
undefined
```

Why?

Students often expect:

```
10
```

But during function execution:

Memory Phase of test()

```
a -> undefined
```

Local variable is created.

Execution:

```
console.log(a);
```

finds local variable first.

Output:

```
undefined
```

Then:

```
a = 20;
```

Visualization

```
Global  
  
a -> 10
```

```
Function test()  
  
a -> undefined
```

Local variable shadows global variable.

Variable Shadowing

Example

```
var name = "Anand";  
  
function demo() {  
    var name = "Rahul";  
  
    console.log(name);  
}  
  
demo();
```

Output:

```
Rahul
```

Local variable takes priority.

Hoisting Interview Questions

Question 1

```
console.log(a);
```



```
var a = 5;
```

Output?

undefined

Question 2

```
show();  
  
function show() {  
    console.log("Hello");  
}
```

Output?

Hello

Question 3

```
show();  
  
var show = function() {  
    console.log("Hello");  
}
```

Output?

TypeError

Question 4

```
console.log(a);  
  
let a = 10;
```

Output?

ReferenceError

Question 5

```
console.log(a);  
  
const a = 10;
```

Output?

ReferenceError

▼ Closures

JavaScript Closures

Closures are one of the most powerful and important concepts in JavaScript. They are heavily used in **React, Node.js, Event Handlers, Timers, Data Hiding, Module Patterns, and Interviews.**

Real Life Analogy

Imagine a student graduates from a college.

```
College  
├─ Library  
├─ Teachers  
└─ Notes
```

After graduation, the student leaves the college.

But the student still remembers:

```
Teacher's Lessons  
Library Knowledge  
Notes
```

Even though the college session has ended.

Similarly:

```
A function remembers  
the variables from its outer scope  
even after the outer function has finished execution.
```

This memory is called **Closure**

What is a Closure?

Definition

A Closure is created when an inner function remembers and accesses variables from its outer function even after the outer function has finished execution.

Simple Example

```
function outer() {  
  
    let message = "Hello Students";  
  
    function inner() {  
        console.log(message);  
    }  
  
    return inner;  
}  
  
const result = outer();
```

```
result();
```

Output:

```
Hello Students
```

What Happened?

Step 1

```
outer();
```

Execution Context Created

Memory:

```
message = "Hello Students"  
inner = function
```

Step 2

```
return inner;
```

Outer function finishes.

Normally students think:

```
message should be destroyed
```

But JavaScript does something special.

Step 3

The returned function keeps a reference to:

```
message = "Hello Students"
```

This preserved memory is called:

```
Closure
```

Visualization

```
outer()  
  
message = "Hello Students"  
  
  ↓  
  
return inner  
  
  ↓  
  
outer removed  
  
  ↓  
  
inner still remembers  
  
message = "Hello Students"
```

Interview Definition

A Closure is:

```
Function  
+  
Its Lexical Environment
```

or

```
Inner Function  
+  
Reference to Outer Variables
```

Example 1: Counter Application

This is one of the most common closure examples.

```
function counter() {  
  
    let count = 0;  
  
    return function () {  
  
        count++;  
  
        console.log(count);  
    };  
}  
  
const increment = counter();  
  
increment();  
increment();  
increment();
```

Output:

```
1  
2  
3
```

Why?

First call:

```
count = 0
```

↓

```
count++
```



```
1
```

Second call:

```
count still exists
```



```
2
```

Third call:

```
count still exists
```



```
3
```

Without Closure:

```
count would become 0 every time
```

Real Life Analogy

Think of a bank account.

```
Account Balance = 1000
```

After depositing:

1200

After depositing again:

1500

Balance is remembered.

Closures work similarly.

Example 2: Private Variable (Data Hiding)

One of the biggest uses of closures.

```
function bankAccount() {  
  
    let balance = 1000;  
  
    return {  
  
        deposit(amount) {  
            balance += amount;  
            console.log("Balance:", balance);  
        },  
  
        withdraw(amount) {  
            balance -= amount;  
            console.log("Balance:", balance);  
        }  
    };  
}  
  
const account = bankAccount();  
  
account.deposit(500);
```



```
account.withdraw(200);
```

Output:

```
Balance: 1500  
Balance: 1300
```

Try this:

```
console.log(account.balance);
```

Output:

```
undefined
```

Why?

Because:

```
balance is private
```

Only closure functions can access it.

Industry Use Case

This concept is used in:

```
Authentication  
Banking Systems  
Payment Gateways  
Shopping Cart
```

to protect sensitive data.

Example 3: Shopping Cart

```
function createCart() {

    let totalItems = 0;

    return function() {

        totalItems++;

        console.log(
            "Items in Cart:",
            totalItems
        );
    };
}

const addItem = createCart();

addItem();
addItem();
addItem();
```

Output:

```
Items in Cart: 1
Items in Cart: 2
Items in Cart: 3
```

Example 4: Function Factory

Closures can generate customized functions.

```
function multiply(x) {

    return function(y) {

        return x * y;
    };
}
```

```
}  
  
const double = multiply(2);  
  
const triple = multiply(3);  
  
console.log(double(5));  
  
console.log(triple(5));
```

Output:

```
10  
15
```

Visualization

```
double  
  
x = 2
```

remembers:

```
x = 2
```

```
triple  
  
x = 3
```

remembers:

```
x = 3
```

This is a closure.

Example 5: setTimeout and Closures

Very important interview example.

```
function greet() {  
  
    let name = "Anand";  
  
    setTimeout(function() {  
  
        console.log(name);  
  
    }, 2000);  
}  
  
greet();
```

Output after 2 seconds:

Anand

Question:

Outer function already finished.

Then how does timeout access name?

Answer:

Closure

The callback remembers the outer variable.

Example 6: Module Pattern

Before ES6 Modules, closures were used to create private members.

```
function userModule() {
```

```
let username = "Admin";

return {

  getUser() {
    return username;
  }
};

const user = userModule();

console.log(user.getUser());
```

Output:

Admin

Closures and Memory

Students often ask:

Why doesn't JavaScript destroy outer variables?

Normally:

Function Finished



Execution Context Removed



Memory Removed

But if an inner function still needs variables:

```
JavaScript keeps them alive
```

This preserved memory becomes Closure Memory.

Most Asked Interview Question

Predict Output

```
function outer() {  
    let count = 0;  
    return function() {  
        count++;  
        console.log(count);  
    };  
}  
  
const fn1 = outer();  
const fn2 = outer();  
  
fn1();  
fn1();  
  
fn2();  
fn2();
```

Output:

```
1  
2
```

```
1  
2
```

Why?

Because:

```
fn1 and fn2 have different closures
```

Each gets its own copy of count.

Famous Interview Question

Predict Output

```
for(var i = 1; i <= 3; i++) {  
    setTimeout(function() {  
        console.log(i);  
    }, 1000);  
}
```

Output:

```
4  
4  
4
```

Why?

By the time timeout executes:

```
Loop already finished
```

and

```
i = 4
```

All callbacks share the same variable.

Solution Using let

```
for(let i = 1; i <= 3; i++) {  
  
    setTimeout(function() {  
  
        console.log(i);  
  
    }, 1000);  
}
```

Output:

```
1  
2  
3
```

Why?

Because:

```
let creates a new block scope  
for every iteration.
```

Each callback gets its own closure.

Advanced Example (Independent Counters)

```
function createCounter() {  
  
    let count = 0;
```



```

    return {
      increment() {
        count++;
        return count;
      },

      decrement() {
        count--;
        return count;
      },

      reset() {
        count = 0;
        return count;
      }
    };
  }

  const counter = createCounter();

  console.log(counter.increment());

  console.log(counter.increment());

  console.log(counter.decrement());

  console.log(counter.reset());

```

Output:

```

1
2
1
0

```

Advantages of Closures

1. Data Privacy

Hide variables from outside world

2. State Management

Remember values between function calls

3. Function Factories

Create customized functions

4. Event Handlers

Buttons remember data

5. Timers

setTimeout
setInterval

6. Module Pattern

Private methods and variables

Disadvantages of Closures

Memory Consumption

Because variables are preserved:

Memory remains occupied

If unnecessary closures are created:

Memory Leaks can occur

Golden Rule

Remember:

```
Closure =  
Function  
+  
Lexical Environment
```

Student Practice Questions

Question 1

Predict Output

```
function outer() {  
  
    let x = 10;  
  
    return function() {  
  
        console.log(x);  
    };  
}  
  
const fn = outer();  
  
fn();
```

Question 2

Predict Output

```
function counter() {  
    let count = 0;  
    return function() {  
        count++;  
        console.log(count);  
    };  
}  
  
const c = counter();  
  
c();  
c();  
c();
```

Question 3

Predict Output

```
for(let i = 1; i <= 3; i++) {  
    setTimeout(() => {  
        console.log(i);  
    }, 1000);  
}
```

Question 4

Predict Output

```
for(var i = 1; i <= 3; i++) {  
  
    setTimeout(() => {  
  
        console.log(i);  
  
    }, 1000);  
}
```

▼ Call, Apply & Bind

JavaScript Call, Apply & Bind

call(), apply(), and bind() are methods that allow us to control the value of the **this** keyword inside a function.

This is one of the most important JavaScript interview topics because it tests your understanding of:

- ✓ Functions
- ✓ Objects
- ✓ **this** Keyword
- ✓ Function Borrowing

Before Learning Call, Apply & Bind

Let's first understand:

What is **this** ?

In JavaScript, **this** refers to the object that is calling the function.

Example:

```
const student = {  
    name: "Rahul",  
  
    greet() {
```

```
        console.log(this.name);  
    }  
};  
  
student.greet();
```

Output:

```
Rahul
```

Here:

```
this === student
```

because `student` called the function.

Real Life Analogy

Imagine a teacher says:

```
"My Name is Anand"
```

Question:

Who is "My"?

Answer depends on who is speaking.

If Anand says it:

```
My = Anand
```

If Rahul says it:

```
My = Rahul
```

Similarly:

this changes depending on
who calls the function

Problem Statement

Suppose we have:

```
const person1 = {  
  name: "Anand"  
};  
  
function introduce() {  
  console.log("My Name is", this.name);  
}
```

Can we use the same function for different objects?

Yes.

This is where:

```
call()  
apply()  
bind()
```

come into the picture.

What is call()?

`call()` immediately invokes a function and allows us to specify what `this` should refer to.

Syntax

```
functionName.call(thisArg, arg1, arg2, arg3);
```

Example 1

```
const person = {  
  name: "Anand"  
};  
  
function greet() {  
  console.log("Hello", this.name);  
}  
  
greet.call(person);
```

Output:

```
Hello Anand
```

What Happened?

Normally:

```
greet();
```

Output:

```
Hello undefined
```

Because:

```
this.name
```

doesn't know which object to use.

But:

```
greet.call(person);
```

forces:


```
this = person
```

Visualization

```
greet()  
  
this -> undefined
```

```
greet.call(person)  
  
this -> person
```

Example 2 (Passing Arguments)

```
const person = {  
  name: "Anand"  
};  
  
function introduce(city, country) {  
  
  console.log(  
    this.name,  
    city,  
    country  
  );  
}  
  
introduce.call(  
  person,  
  "Delhi",  
  "India"  
);
```

Output:

Anand Delhi India

What is apply()?

`apply()` is almost identical to `call()`.

Difference:

`call()` -> arguments separately

`apply()` -> arguments inside array

Syntax

```
functionName.apply(thisArg, [arg1, arg2]);
```

Example

```
const person = {
  name: "Anand"
};

function introduce(city, country) {

  console.log(
    this.name,
    city,
    country
  );
}

introduce.apply(
  person,
```

```
    ["Delhi", "India"]  
);
```

Output:

```
Anand Delhi India
```

Comparison

call()

```
introduce.call(  
    person,  
    "Delhi",  
    "India"  
);
```

apply()

```
introduce.apply(  
    person,  
    ["Delhi", "India"]  
);
```

Real Life Analogy

Imagine ordering pizza.

call()

```
Pizza  
Coke  
Burger
```

separately.

apply()

```
[Pizza, Coke, Burger]
```

inside one box.

What is bind()?

`bind()` does not immediately execute the function.

Instead:

```
Creates a new function  
with fixed this value
```

Syntax

```
const newFunction =  
  oldFunction.bind(thisArg);
```

Example

```
const person = {  
  name: "Anand"  
};  
  
function greet() {  
  console.log(  
    "Hello",  
    this.name  
  );  
}  
  
const newFunction =  
  greet.bind(person);
```

```
newFunction();
```

Output:

```
Hello Anand
```

Important Difference

call()

Executes immediately.

```
greet.call(person);
```

bind()

Returns new function.

```
const fn =  
    greet.bind(person);  
  
fn();
```

Visualization

```
call()
```

```
Create  
Execute  
Done
```

```
bind()
```

```
Create
```

Store
Execute Later

Example with Arguments

```
const person = {  
  name: "Anand"  
};  
  
function introduce(city) {  
  
  console.log(  
    this.name,  
    city  
  );  
}  
  
const fn =  
  introduce.bind(  
    person,  
    "Delhi"  
  );  
  
fn();
```

Output:

Anand Delhi

call vs apply vs bind

Feature	call()	apply()	bind()
Executes Immediately	✓	✓	✗
Returns Function	✗	✗	✓
Arguments	Separate	Array	Separate

Feature	call()	apply()	bind()
Changes this	✓	✓	✓

Function Borrowing

One of the biggest use cases.

Suppose:

```
const student1 = {
  name: "Rahul",

  greet() {
    console.log(
      "Hello",
      this.name
    );
  }
};

const student2 = {
  name: "Anand"
};
```

Can student2 use greet()?

Yes.

Using call()

```
student1.greet.call(student2);
```

Output:

```
Hello Anand
```

Visualization

```
student1 owns greet()

student2 borrows greet()
```

This is called:

Function Borrowing

Example: Company Employees

```
const employee1 = {
  name: "Rahul"
};

const employee2 = {
  name: "Anand"
};

function showRole(role) {

  console.log(
    this.name,
    role
  );
}

showRole.call(
  employee1,
  "Developer"
);

showRole.call(
  employee2,
  "Trainer"
);
```


Output:

```
Rahul Developer  
Anand Trainer
```

Real World Use Case

Event Handlers

```
button.addEventListener(  
  "click",  
  user.login.bind(user)  
);
```

Why?

Because:

```
bind()  
ensures correct this value
```

when the event occurs.

Interview Question 1

Predict Output

```
const person = {  
  name: "Rahul"  
};  
  
function greet() {  
  console.log(this.name);  
}  
  
greet.call(person);
```

Output:

Rahul

Interview Question 2

Predict Output

```
const person = {  
  name: "Rahul"  
};  
  
function greet(city) {  
  console.log(  
    this.name,  
    city  
  );  
}  
  
greet.apply(  
  person,  
  ["Delhi"]  
);
```

Output:

Rahul Delhi

Interview Question 3

Predict Output

```
const person = {  
  name: "Rahul"  
};  
  
function greet() {
```

```
        console.log(this.name);
    }

    const fn =
        greet.bind(person);

    fn();
```

Output:

Rahul

Interview Question 4

Predict Output

```
const person = {
    name: "Anand"
};

function greet() {
    console.log(this.name);
}

const fn =
    greet.bind(person);

greet.call({
    name: "Rahul"
});

fn();
```

Output:

Rahul

Anand

Advanced Example

```
function calculate(a, b) {  
  return a + b;  
}  
  
const add10 =  
  calculate.bind(  
    null,  
    10  
  );  
  
console.log(add10(5));
```

Output:

15

Why?

`add10(5)`

becomes:

`calculate(10, 5)`

This is called:

Partial Application

▼ Constructor Function

Constructor Function

Constructor Functions are one of the most important JavaScript concepts because they teach students **how objects are created internally**, how the **new keyword works**, and prepare them for **Prototype, Classes, and OOP**.

Why Do We Need Constructor Functions?

Imagine you are building a University Management System.

You need:

```
1000 Students
500 Teachers
200 Courses
```

Without Constructor Functions:

```
const student1 = {
  name: "Rahul",
  age: 20,
  course: "BCA"
};

const student2 = {
  name: "Amit",
  age: 21,
  course: "MCA"
};

const student3 = {
  name: "Rohit",
  age: 22,
  course: "B.Tech"
};
```

Problem:

```
Code Repetition
Hard to Maintain
```

Not Scalable

We need a blueprint.

Real Life Analogy

Think of a Cookie Cutter.

```
Cookie Cutter
  ↓
Cookie 1
Cookie 2
Cookie 3
Cookie 100
```

The cutter is created once.

Many cookies are created from it.

Similarly:

```
Constructor Function
  ↓
Object 1
Object 2
Object 3
Object 100
```

What is a Constructor Function?

A Constructor Function is a special function used to create multiple objects having the same structure and behavior.

First Constructor Function

```
function Student(name, age, course) {

    this.name = name;
    this.age = age;
    this.course = course;
```

```
}
```

Notice:

```
Student
```

starts with a capital letter.

This is a JavaScript convention.

Creating Objects

```
const student1 =  
  new Student(  
    "Rahul",  
    20,  
    "BCA"  
  );  
  
const student2 =  
  new Student(  
    "Amit",  
    21,  
    "MCA"  
  );
```

Output:

```
console.log(student1);
```

```
{  
  name: "Rahul",  
  age: 20,  
  course: "BCA"  
}
```

The Biggest Question

Students usually ask:

Where did this object come from?

We never wrote:

```
{}
```

But still we got an object.

Why?

Because of the:

```
new keyword
```

What Happens Internally?

When JavaScript sees:

```
new Student("Rahul", 20);
```

It performs 4 hidden steps.

Step 1: Create Empty Object

JavaScript secretly creates:

```
{}
```

Step 2: Bind this

```
this = {}
```

Now:

```
this.name = "Rahul";
```


becomes

```
{  
  name: "Rahul"  
}
```

Step 3: Execute Constructor

```
this.age = 20;
```

becomes

```
{  
  name: "Rahul",  
  age: 20  
}
```

Step 4: Return Object

JavaScript automatically returns:

```
{  
  name: "Rahul",  
  age: 20  
}
```

Visualization

```
new Student("Rahul")
```

↓

```
Create Empty Object
```

```
{}
```

↓

```
Bind this
```

```
this = {}
```

↓

Execute Constructor

```
this.name = Rahul
```

```
this.age = 20
```

↓

Return Object

```
{  
  name: Rahul,  
  age: 20  
}
```

Understanding this

Consider:

```
function Student(name) {  
  
    this.name = name;  
}
```

Question:

Who is `this` ?

Answer:

The object currently being created

Example:

```
const s1 =  
    new Student("Rahul");
```

Here:

```
this === s1
```

Example 1: Employee System

```
function Employee(  
    name,  
    department,  
    salary  
) {  
  
    this.name = name;  
    this.department = department;  
    this.salary = salary;  
}
```

Create employees:

```
const emp1 =  
    new Employee(  
        "Rahul",  
        "IT",  
        50000  
    );  
  
const emp2 =  
    new Employee(  
        "Amit",  
        "HR",  
        45000  
    );
```

Output:

```
console.log(emp1);
```

```
{  
  name: "Rahul",  
  department: "IT",  
}
```

```
    salary: 50000  
  }
```

Example 2: Mobile Shop

```
function Mobile(  
  brand,  
  model,  
  price  
) {  
  
  this.brand = brand;  
  this.model = model;  
  this.price = price;  
}
```

Create Objects

```
const m1 =  
  new Mobile(  
    "Apple",  
    "iPhone 16",  
    90000  
  );  
  
const m2 =  
  new Mobile(  
    "Samsung",  
    "S26",  
    85000  
  );
```

Adding Methods

Students usually write:

```
function Student(name) {  
  
    this.name = name;  
  
    this.greet = function() {  
  
        console.log(  
            "Hello",  
            this.name  
        );  
    };  
}
```

```
const s1 =  
    new Student("Rahul");  
  
s1.greet();
```

Hello Rahul

1000 Students

```
function greet(){}
```

Huge memory waste.

Visualization

```
Student1
  greet()

Student2
  greet()

Student3
  greet()

Student4
  greet()
```

Same function copied repeatedly.

Better Solution → Prototype

Instead of creating methods inside each object:

```
function Student(name) {

  this.name = name;
}
```

Add methods to Prototype.

```
Student.prototype.greet =
function() {

  console.log(
    "Hello",
    this.name
  );
};
```

Now:

```
const s1 =  
  new Student("Rahul");  
  
const s2 =  
  new Student("Amit");
```

Both share:

```
greet()
```

Visualization

```
graph TD; s1 --> P[Prototype]; s2 --> P;
```

A diagram illustrating the prototype chain. Two variables, s1 and s2, are shown on the left. From s1, a line goes down and to the right, then up and to the right to a central node labeled 'Prototype'. From s2, a line goes up and to the right, then down and to the right to the same 'Prototype' node.

Shared Method:

```
greet()
```

Only one copy exists.

Constructor Function + Multiple Methods

```
function Car(  
  brand,  
  model  
) {  
  
  this.brand = brand;  
  this.model = model;  
}
```

Methods

```
Car.prototype.start =  
function() {  
  
    console.log(  
        this.brand +  
        " Started"  
    );  
};  
  
Car.prototype.stop =  
function() {  
  
    console.log(  
        this.brand +  
        " Stopped"  
    );  
};
```

Usage

```
const car1 =  
    new Car(  
        "BMW",  
        "X5"  
    );  
  
car1.start();  
  
car1.stop();
```

Output:

```
BMW Started  
BMW Stopped
```


Constructor Function Returning Values

Most students don't know this interview question.

Example:

```
function Person(name) {  
    this.name = name;  
    return "Hello";  
}  
  
const p =  
    new Person("Rahul");  
  
console.log(p);
```

Output:

```
{  
  name: "Rahul"  
}
```

Why?

Because:

Primitive return values are ignored

Example

```
function Person(name) {  
    this.name = name;  
    return {  
        city: "Delhi"  
    }  
}
```

```
};  
}  
  
const p =  
    new Person("Rahul");  
  
console.log(p);
```

Output:

```
{  
  city: "Delhi"  
}
```

Why?

Because:

```
Returned Object overrides  
the default object
```

What Happens If We Forget new?

Example

```
function Student(name) {  
  
    this.name = name;  
}  
  
const s1 =  
    Student("Rahul");
```

Wrong.

Why?

Because:

```
No object is created
```

and

```
this will not refer  
to a new object
```

Always use:

```
new Student("Rahul");
```

Built-In Constructor Functions

JavaScript itself uses constructor functions.

Example:

```
const date =  
    new Date();  
  
console.log(date);
```

Array

```
const arr =  
    new Array(  
        1,  
        2,  
        3  
    );
```

Object

```
const obj =  
    new Object();
```

These are built-in constructor functions.

Constructor Function vs Object Literal

Object Literal

```
const student = {  
  name: "Rahul"  
};
```

Best for:

Single Object

Constructor Function

```
new Student();
```

Best for:

Multiple Similar Objects

Constructor Function vs Class

Constructor Function

```
function Student(name) {  
  
  this.name = name;  
}
```

ES6 Class

```
class Student {

    constructor(name) {

        this.name = name;
    }
}
```

Both create objects.

Class syntax is cleaner.

Industry Example

Imagine an E-Commerce Application.

```
function Product(
    id,
    name,
    price
) {

    this.id = id;
    this.name = name;
    this.price = price;
}
```

Products:

```
const p1 =
    new Product(
        1,
        "Laptop",
        60000
    );

const p2 =
    new Product(
        2,
```

```
        "Phone",  
        25000  
    );
```

This is exactly how large applications model data.

Interview Questions

Predict Output

```
function Student(name) {  
  
    this.name = name;  
}  
  
const s1 =  
    new Student(  
        "Rahul"  
    );  
  
console.log(  
    s1.name  
);
```

Output:

```
Rahul
```

Predict Output

```
function Test() {  
  
    this.a = 10;  
  
    return 100;  
}
```

```
const obj =  
    new Test();  
  
console.log(obj);
```

Output:

```
{  
  a: 10  
}
```

Predict Output

```
function Test() {  
  
    this.a = 10;  
  
    return {  
        b: 20  
    };  
}  
  
const obj =  
    new Test();  
  
console.log(obj);
```

Output:

```
{  
  b: 20  
}
```

▼ ES6 Features

JavaScript ES6 (ECMAScript 2015) Features

ES6 (ECMAScript 2015) was the biggest update in JavaScript history. It introduced many modern features that made JavaScript cleaner, more readable, and easier to maintain.

Before ES6, JavaScript code was often verbose and difficult to manage. ES6 solved many of these problems.

Why Was ES6 Introduced?

Before ES6

```
var name = "Rahul";

function greet(name) {
  name = name || "Guest";

  console.log("Hello " + name);
}

greet();
```

Problems:

```
Too much code
Difficult to read
Error-prone
```

After ES6

```
const name = "Rahul";

function greet(name = "Guest") {
  console.log(`Hello ${name}`);
}

greet();
```


Cleaner, shorter, and easier to understand.

Topics Covered

1. let
 2. const
 3. Template Literals
 4. Arrow Functions
 5. Default Parameters
 6. Rest Parameters
 7. Spread Operator
 8. Destructuring
 9. Enhanced Object Literals
 10. Optional Chaining
 11. Nullish Coalescing Operator
 12. Modules
 13. Classes
 14. for...of Loop
-

1. let Keyword

Before ES6:

```
var age = 20;
```

Problem with `var`:

```
{  
  var age = 20;  
}  
  
console.log(age);
```

Output:

```
20
```

Because:

```
var is NOT block scoped
```

Solution: let

```
{  
  let age = 20;  
}  
  
console.log(age);
```

Output:

```
ReferenceError
```

Because:

```
let is block scoped
```

Real Life Analogy

Think of a classroom.

```
Classroom  
|  
|-- Student A  
|-- Student B
```

Information inside the classroom should remain inside the classroom.

That is exactly how `let` works.

2. const Keyword

Used when value should not change.

```
const PI = 3.14;  
  
PI = 4;
```

Output:

```
TypeError
```

Real Example

```
const companyName = "Google";  
  
const apiUrl = "https://api.example.com";
```

These values rarely change.

Important Interview Question

```
const user = {  
  name: "Rahul"  
};  
  
user.name = "Amit";
```

Allowed?

Answer:

```
YES
```

Output:

```
{  
  name: "Amit"  
}
```

Why?

Because:

```
const prevents reassignment  
NOT object modification
```

3. Template Literals

Before ES6:

```
const name = "Rahul";  
  
console.log("Hello " + name);
```

ES6

```
const name = "Rahul";  
  
console.log(`Hello ${name}`);
```

Output:

```
Hello Rahul
```

Multiple Variables

```
const name = "Rahul";  
const age = 22;
```

```
console.log(  
  `My name is ${name} and I am ${age} years old`  
);
```

Output:

```
My name is Rahul and I am 22 years old
```

Multiline Strings

Before ES6:

```
const msg =  
"Hello\n"  
"World";
```

ES6:

```
const msg = `  
Hello  
World  
`;
```

4. Arrow Functions

Before ES6:

```
function add(a, b) {  
  return a + b;  
}
```

ES6:

```
const add = (a, b) => {  
  return a + b;  
};
```

Short Form

```
const add = (a, b) => a + b;
```

Output:

```
console.log(add(5, 10));
```

15

One Parameter

```
const square = num => num * num;
```

No Parameters

```
const greet = () => "Hello";
```

Real World Example

```
const users = ["Rahul", "Amit", "Anand"];  
  
users.forEach(user =>  
  console.log(user)  
);
```

Arrow Function and this

Normal Function:

```
const person = {  
  name: "Rahul",  
  greet: function() {  
    console.log(this.name);  
  }  
};  
  
person.greet();
```

Output:

Rahul

Arrow Function:

```
const person = {  
  name: "Rahul",  
  greet: () => {  
    console.log(this.name);  
  }  
};  
  
person.greet();
```

Output:

undefined

Why?

Arrow Functions do not have their own this.

This is a very common interview question.

5. Default Parameters

Before ES6:

```
function greet(name) {  
    name = name || "Guest";  
    console.log(name);  
}
```

ES6:

```
function greet(name = "Guest") {  
    console.log(name);  
}
```

Usage:

```
greet();
```

Output:

```
Guest
```

6. Rest Parameters (...)

Used when number of arguments is unknown.

```
function total(...numbers) {
```



```
    console.log(numbers);  
  }  
  
  total(10, 20, 30, 40);
```

Output:

```
[10, 20, 30, 40]
```

Sum Example

```
function sum(...nums) {  
  
  return nums.reduce(  
    (acc, num) => acc + num,  
    0  
  );  
}  
  
console.log(sum(10, 20, 30));
```

Output:

```
60
```

Real Life Analogy

Imagine a shopping cart.

```
1 item  
5 items  
20 items
```

Rest parameter collects everything into one array.

7. Spread Operator (...)

Looks similar to Rest but behaves differently.

Array Copy

```
const arr1 = [1,2,3];  
  
const arr2 = [...arr1];
```

Output:

```
[1,2,3]
```

Array Merge

```
const frontend =  
  ["HTML", "CSS"];  
  
const backend =  
  ["Node", "MongoDB"];  
  
const stack =  
  [  
    ...frontend,  
    ...backend  
  ];
```

Output:

```
[  
  "HTML",  
  "CSS",  
  "Node",  
  "MongoDB"  
]
```

Object Copy

```
const user = {  
  name: "Rahul"  
};  
  
const copy = {  
  ...user  
};
```

Rest vs Spread

Rest

```
function demo(...args){}
```

Collects values.

Spread

```
demo(...arr);
```

Expands values.

8. Destructuring

Used to extract values easily.

Array Destructuring

```
const colors =  
  ["Red", "Blue", "Green"];  
  
const [a, b, c] = colors;
```

```
console.log(a);
```

Output:

Red

Object Destructuring

```
const user = {  
  name: "Rahul",  
  age: 22  
};  
  
const {  
  name,  
  age  
} = user;
```

Output:

Rahul
22

Rename Variable

```
const user = {  
  name: "Rahul"  
};  
  
const {  
  name: studentName  
} = user;
```

```
console.log(studentName);
```

Real World Example

API Response:

```
const response = {  
  id: 1,  
  title: "Laptop",  
  price: 50000  
};  
  
const {  
  title,  
  price  
} = response;
```

9. Enhanced Object Literals

Before ES6:

```
const name = "Rahul";  
  
const user = {  
  name: name  
};
```

ES6:

```
const name = "Rahul";  
  
const user = {  
  name  
};
```

Methods

```
const user = {  
  greet() {  
    console.log("Hello");  
  }  
};
```

10. Optional Chaining (?.)

Problem:

```
const user = {};  
  
console.log(  
  user.address.city  
);
```

Output:

TypeError

Solution:

```
console.log(  
  user.address?.city  
);
```

Output:

undefined

Real World Example

API responses often have missing data.

```
const user = {  
  name: "Rahul"  
};  
  
console.log(  
  user.profile?.email  
);
```

Safe access.

11. Nullish Coalescing Operator (??)

Provides default value only when value is:

```
null  
undefined
```

Example

```
const username = null;  
  
console.log(  
  username ?? "Guest"  
);
```

Output:

```
Guest
```

Difference from ||

```
const count = 0;  
  
console.log(  
  count || 1  
);
```

```
    count || 10  
);
```

Output:

```
10
```

Wrong.

Using ??

```
console.log(  
    count ?? 10  
);
```

Output:

```
0
```

Correct.

12. Modules

File: math.js

```
export function add(a,b){  
    return a+b;  
}
```

File: app.js

```
import { add }  
from "./math.js";  
  
console.log(  
    add(10,20)  
);
```


Output:

```
30
```

Benefits:

```
Reusable Code  
Maintainable Code  
Industry Standard
```

13. Classes

ES6 introduced cleaner syntax for OOP.

Before:

```
function Student(name) {  
  this.name = name;  
}
```

ES6:

```
class Student {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    console.log(  
      this.name  
    );  
  }  
}
```

Usage

```
const s1 =  
new Student("Rahul");  
  
s1.greet();
```

14. for...of Loop

Used for arrays.

```
const colors =  
["Red", "Blue", "Green"];  
  
for(const color of colors) {  
  
    console.log(color);  
}
```

Output:

```
Red  
Blue  
Green
```

Mini Industry Example

```
const products = [  
    {  
        id:1,  
        name:"Laptop",  
        price:50000  
    },  
    {  
        id:2,  
        name:"Phone",  
        price:30000  
    }  
];
```

```
const discountedProducts =  
products.map(product => ({  
  ...product,  
  price: product.price - 5000  
}));  
  
console.log(discountedProducts);
```

▼ JSON, Local, & Session Storage

JavaScript JSON, Local Storage & Session Storage

JSON, Local Storage, and Session Storage are used extensively in modern web applications. Every React, Angular, Vue, Node.js, and MERN Stack application uses these concepts for storing and transferring data.

Part 1: JSON (JavaScript Object Notation)

Before learning Local Storage, students must understand JSON because:

Local Storage stores only strings.

When we want to store objects or arrays, we convert them into JSON.

Real Life Analogy

Imagine you are sending a parcel from Delhi to Mumbai.

You cannot send:

```
{  
  name: "Rahul",  
  age: 25  
}
```

directly.

Instead, you pack it into a standard format.

Similarly:

```
JSON is a standard format  
for exchanging data.
```

What is JSON?

JSON stands for:

```
JavaScript Object Notation
```

It is a lightweight format used to exchange data between:

```
Frontend ↔ Backend  
Frontend ↔ API  
Server ↔ Database
```

Example JSON

```
{  
  "name": "Rahul",  
  "age": 25,  
  "city": "Delhi"  
}
```

Notice:

JSON Rules

Keys must be inside double quotes.

```
{  
  "name": "Rahul"  
}
```

Valid.

```
{  
  name: "Rahul"  
}
```

Invalid JSON.

JavaScript Object vs JSON

JavaScript Object

```
const student = {  
  name: "Rahul",  
  age: 25  
};
```

JSON

```
{  
  "name": "Rahul",  
  "age": 25  
}
```

Why JSON?

Suppose frontend sends:

```
const user = {  
  name: "Rahul",  
  age: 25  
};
```

Backend cannot understand JavaScript objects directly.

So we convert:

Object
↓
JSON
↓
Network
↓
Server

JSON.stringify()

Converts:

Object → JSON String

Example

```
const student = {  
  name: "Rahul",  
  age: 25  
};  
  
const jsonData =  
  JSON.stringify(student);  
  
console.log(jsonData);
```

Output:

```
'{"name": "Rahul", "age": 25}'
```

Notice:

Output is STRING

Why stringify?

Because:

APIs
Local Storage
Session Storage

store data as strings.

JSON.parse()

Converts:

JSON String → Object

Example

```
const jsonData =  
  '{"name": "Rahul", "age": 25}';  
  
const user =  
  JSON.parse(jsonData);  
  
console.log(user);
```

Output:

```
{  
  name: "Rahul",  
  age: 25  
}
```

Visual Flow

```
Object  
  ↓  
JSON.stringify()  
  ↓  
JSON String  
  ↓
```

```
JSON.parse()  
↓  
Object
```

Industry Example: API Response

Suppose backend returns:

```
{  
  "id":1,  
  "name":"Laptop",  
  "price":50000  
}
```

Frontend receives JSON.

Convert:

```
const product =  
JSON.parse(response);
```

Now:

```
console.log(product.price);
```

Output:

```
50000
```

Part 2: Local Storage

What is Local Storage?

Local Storage allows websites to store data inside the browser.

Data remains available even after:

```
Browser Closed  
Tab Closed
```


Computer Restarted

Real Life Analogy

Think of Local Storage as:

A Locker

You put something inside.

Even if you leave the office and return tomorrow:

The locker still contains data.

Local Storage Capacity

Approximately:

5 MB to 10 MB

depending on browser.

Local Storage Methods

Method	Purpose
setItem()	Store Data
getItem()	Retrieve Data
removeItem()	Remove Data
clear()	Remove Everything

Storing Data

```
localStorage.setItem(  
  "name",
```

```
    "Rahul"  
  );
```

Retrieving Data

```
const name =  
  localStorage.getItem("name");  
  
console.log(name);
```

Output:

```
Rahul
```

Removing Data

```
localStorage.removeItem("name");
```

Clearing Entire Storage

```
localStorage.clear();
```

Industry Example: Dark Theme

Suppose user selects:

```
Dark Mode
```

Store preference.

```
localStorage.setItem(  
  "theme",  
  "dark"  
);
```

Next time user visits:

```
const theme =  
localStorage.getItem("theme");  
  
if(theme === "dark") {  
  
    document.body.classList.add(  
        "dark"  
    );  
}
```

User preference is remembered.

Problem

Can we store objects?

```
const user = {  
    name: "Rahul"  
};  
  
localStorage.setItem(  
    "user",  
    user  
);
```

Output:

```
[object Object]
```

Wrong.

Solution

Use JSON.stringify()

```
const user = {
  name: "Rahul",
  age: 25
};

localStorage.setItem(
  "user",
  JSON.stringify(user)
);
```

Retrieving Object

```
const user =
JSON.parse(
  localStorage.getItem("user")
);

console.log(user);
```

Output:

```
{
  name: "Rahul",
  age: 25
}
```

Industry Example: Shopping Cart

E-commerce websites use Local Storage.

```
const cart = [
  {
    id: 1,
    product: "Laptop"
  },
  {
    id: 2,
```

```
        product: "Phone"
    }
];

localStorage.setItem(
    "cart",
    JSON.stringify(cart)
);
```

Retrieve Cart

```
const cartData =
JSON.parse(
    localStorage.getItem("cart")
);

console.log(cartData);
```

User closes browser.

Returns tomorrow.

Cart still exists.

Industry Example: Notes App

Store notes.

```
const notes = [
    "Learn JS",
    "Learn React"
];

localStorage.setItem(
    "notes",
    JSON.stringify(notes)
);
```

Part 3: Session Storage

What is Session Storage?

Session Storage stores data only for the current browser tab/session.

Data disappears when:

```
Tab Closed  
Browser Window Closed
```

Real Life Analogy

Think of Session Storage as:

```
Hotel Room
```

While staying:

```
Your belongings exist.
```

When checkout happens:

```
Everything removed.
```

Session Storage Methods

Exactly same as Local Storage.

```
sessionStorage.setItem();  
sessionStorage.getItem();  
sessionStorage.removeItem();  
sessionStorage.clear();
```

Example

```
sessionStorage.setItem(  
  "username",
```

```
    "Rahul"  
  );
```

Retrieve

```
const username =  
  sessionStorage.getItem(  
    "username"  
  );
```

Industry Example: OTP Verification

User logs in.

Store:

```
sessionStorage.setItem(  
  "otpVerified",  
  "true"  
);
```

If tab closes:

OTP Session Ends

Industry Example: Multi-Step Form

Suppose:

```
Step 1 → Personal Details  
Step 2 → Address  
Step 3 → Payment
```

Store progress:

```
sessionStorage.setItem(  
  "currentStep",
```

```
2  
);
```

If user refreshes:

```
Continue from Step 2
```

If tab closes:

```
Start Again
```

Local Storage vs Session Storage

Feature	Local Storage	Session Storage
Storage Limit	~5MB	~5MB
Survives Browser Restart	✓	✗
Survives Tab Close	✓	✗
Available Across Tabs	✓	✗
Use Case	User Preferences	Temporary Session Data

Industry Scenarios

Local Storage

Store:

```
Theme Preference  
Language Preference  
Shopping Cart  
Notes  
Draft Articles  
User Settings
```

Session Storage

Store:

OTP Verification
Multi-Step Forms
Temporary Login Data
Checkout Progress

Complete Example: Login User

Store User

```
const user = {  
  id: 1,  
  name: "Rahul",  
  role: "Admin"  
};  
  
localStorage.setItem(  
  "user",  
  JSON.stringify(user)  
);
```

Read User

```
const user =  
JSON.parse(  
  localStorage.getItem("user")  
);  
  
console.log(user.name);
```

Output:

Rahul

Logout User

```
localStorage.removeItem("user");
```

▼ Asynchronous Programming

JavaScript Asynchronous Programming

Asynchronous JavaScript is the foundation of modern web development. Without understanding it properly, students will struggle with:

- Promises
- Async/Await
- Fetch API
- React
- Node.js
- API Calls
- Real-Time Applications

This is one of the **most important JavaScript topics** and should be taught carefully.

Why Do We Need Asynchronous Programming?

Real Life Example

Imagine you order food from Zomato.

Synchronous Behavior

```
Order Food
↓
Wait 30 Minutes
↓
Receive Food
↓
Continue Your Work
```

Problem:

You are blocked for 30 minutes.

Asynchronous Behavior

```
Order Food
↓
Continue Watching Movie
↓
Continue Working
↓
Food Arrives Later
```

Much better.

This is exactly how JavaScript handles long-running tasks.

What is Synchronous JavaScript?

By default, JavaScript is synchronous.

This means:

```
One Task At A Time
Line By Line Execution
```

Example:

```
console.log("Start");

console.log("Learning JavaScript");

console.log("End");
```

Output:

```
Start
Learning JavaScript
End
```

Execution:

```
Start
↓
Learning JavaScript
↓
End
```

JavaScript waits for one task to complete before starting the next.

The Problem with Synchronous Execution

Consider:

```
console.log("Start");

for(let i = 0; i < 10000000000; i++) {
}

console.log("End");
```

Output:

```
Start
(wait several seconds)
End
```

The browser freezes.

Why?

Because JavaScript is busy executing the loop.

Nothing else can run.

What is Asynchronous JavaScript?

Asynchronous programming allows JavaScript to start a task and continue executing other code without waiting for that task to finish.

Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Task Completed");
}, 3000);

console.log("End");
```

Output:

```
Start
End
Task Completed
```

Notice:

```
JavaScript did NOT wait 3 seconds.
```

Most Important Interview Question

Is JavaScript Synchronous or Asynchronous?

Answer:

```
JavaScript is Single-Threaded and Synchronous by Nature.

But it can perform asynchronous operations using Browser APIs.
```

This answer is commonly expected in interviews.

What Does Single-Threaded Mean?

JavaScript has:

```
One Call Stack
```

Only one task can execute at a time.

Real Life Analogy

Imagine one chef.

```
One Chef
↓
One Order At A Time
```

The chef cannot cook 10 dishes simultaneously.

Similarly:

```
One JavaScript Thread
↓
One Task At A Time
```

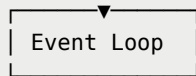
JavaScript Runtime Architecture

To understand asynchronous programming, students must understand:

1. Call Stack
2. Web APIs
3. Callback Queue
4. Event Loop
5. Microtask Queue

The Big Picture





Call Stack

The Call Stack executes JavaScript code.

Example:

```
function one() {  
  two();  
}  
  
function two() {  
  three();  
}  
  
function three() {  
  console.log("Done");  
}  
  
one();
```

Execution:

Stack

three()
two()
one()

After execution:

Removed
Removed
Removed

Rule:

LIFO
Last In First Out

Browser APIs

JavaScript itself cannot perform:

setTimeout
DOM Events
Fetch Requests
Geolocation

The browser provides these features.

These are called:

Web APIs

Example

```
setTimeout(() => {  
  console.log("Hello");  
}, 2000);
```

What happens?

Call Stack
↓
Browser API
↓
Timer Starts
↓
JavaScript Continues

Callback Queue

When a task finishes inside Web APIs:

Task
Completed

Its callback is placed into:

Callback Queue

Example

```
setTimeout(() => {  
  console.log("Done");  
}, 2000);
```

After 2 seconds:

Callback Queue

console.log("Done")

Event Loop

The Event Loop continuously checks:

Is Call Stack Empty?

If YES:

Move callback
from Queue
to Stack

Visualization

Call Stack Empty
↓
Event Loop Checks

↓
Moves Callback
↓
Execute Callback

Complete setTimeout Example

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Timer Finished");  
}, 2000);  
  
console.log("End");
```

Step 1

Output

Start

Step 2

setTimeout sent to Web API

Timer starts.

Step 3

Output

End

Step 4

After 2 seconds:

```
Callback Queue
```

```
Timer Finished
```

Step 5

Event Loop:

```
Call Stack Empty?  
YES
```



Move callback.

Final Output

```
Start  
End  
Timer Finished
```

Callback Functions

A callback is a function passed as an argument to another function.

Example:

```
function greet(name, callback) {  
  
    console.log("Hello", name);  
  
    callback();  
}  
  
function sayBye() {  
  
    console.log("Goodbye");  
}
```

```
greet("Rahul", sayBye);
```

Output:

```
Hello Rahul  
Goodbye
```

Real World Example

```
loginUser(() => {  
  
    loadProfile(() => {  
  
        loadOrders(() => {  
  
            processPayment();  
  
        });  
  
    });  
  
});
```

This leads to:

```
Callback Hell
```

Callback Hell

Example:

```
setTimeout(() => {  
  
    console.log("Step 1");  
  
    setTimeout(() => {
```

```
    console.log("Step 2");

    setTimeout(() => {

        console.log("Step 3");

    }, 1000);

}, 1000);

}, 1000);
```

Visualization

```
Step1
  ↓
  Step2
    ↓
    Step3
```

Looks like:

Pyramid of Doom

Problems:

Difficult to Read
Difficult to Debug
Difficult to Maintain

Microtask Queue

Many students know Callback Queue.

Few know:

Microtask Queue

Microtask Queue has higher priority.

Examples:

```
Promise.then()  
catch()  
finally()  
queueMicrotask()
```

Two Queues

1. Callback Queue
2. Microtask Queue

Priority:

```
Microtask Queue  
    ↑  
Higher Priority  
    ↑  
Callback Queue
```

Most Important Interview Example

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Timeout");  
}, 0);  
  
Promise.resolve()  
    .then(() => {  
        console.log("Promise");  
    });  
  
console.log("End");
```

Output:

```
Start
End
Promise
Timeout
```

Students often expect:

```
Start
End
Timeout
Promise
```

Wrong.

Why?

Because:

```
Promise Callback
↓
Microtask Queue
```

and

```
setTimeout
↓
Callback Queue
```

Microtask Queue executes first.

Detailed Flow

Step 1

```
Start
```

Step 2

```
setTimeout
```

moves to Web API.

Step 3

```
Promise.then()
```

moves to:

```
Microtask Queue
```

Step 4

```
End
```

printed.

Current Queues:

```
Microtask Queue
-----
Promise

Callback Queue
-----
Timeout
```

Event Loop:

```
Process Microtasks First
```

Output:

Promise

Then:

Timeout

Final Output

Start
End
Promise
Timeout

Common Browser APIs

Timer APIs

```
setTimeout()  
setInterval()
```

DOM APIs

```
click  
submit  
keyup  
change
```

Fetch API

```
fetch()
```

Geolocation

```
navigator.geolocation
```

Storage APIs

```
localStorage  
sessionStorage
```

Real Industry Example

Fetching Products

```
console.log("Loading Products...");  
  
setTimeout(() => {  
  
    console.log(  
        "Products Loaded"  
    );  
  
}, 3000);  
  
console.log(  
    "Render UI"  
);
```

Output:

```
Loading Products...  
Render UI  
Products Loaded
```

This is exactly how websites load APIs while keeping the UI responsive.

▼ Promises

JavaScript Promises

Promises were introduced to solve one of the biggest problems in JavaScript:

Callback Hell

Promises make asynchronous code:

- ✓ Easier to Read
- ✓ Easier to Maintain
- ✓ Easier to Debug
- ✓ Easier to Chain Multiple Async Operations

Why Do We Need Promises?

Let's first understand the problem.

Imagine an E-Commerce Website.

Steps:

1. Login User
2. Fetch User Profile
3. Fetch Orders
4. Process Payment

Using Callbacks:

```
loginUser(function() {  
  
    getUserProfile(function() {  
  
        getOrders(function() {  
  
            processPayment(function() {  
  
                console.log("Done");  
  
            });  
  
        });  
  
    });  
  
});
```

```
    });  
  
});
```

Problem

This structure becomes:

```
login  
└─ profile  
    └─ orders  
        └─ payment
```

Looks like:

Pyramid of Doom

Also called:

Callback Hell

Real Life Analogy

Imagine ordering food online.

```
Order Food  
↓  
Food Being Prepared  
↓  
Food Out For Delivery  
↓  
Food Delivered
```

The result will come in the future.

You don't know exactly when.

This is exactly how a Promise works.

What is a Promise?

Definition

A Promise is an object that represents the eventual completion or failure of an asynchronous operation.

Simple Definition

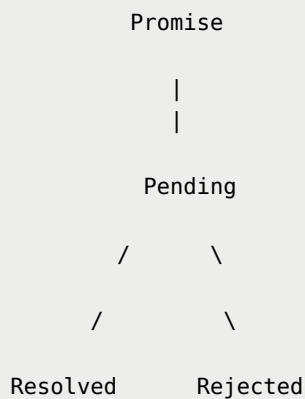
A Promise is a placeholder for a value that will be available in the future.

Promise States

Every Promise has 3 states.

```
Pending
Resolved (Fulfilled)
Rejected
```

Visualization



Real Life Example

Food Delivery

```
Order Placed
```



Pending



Food Delivered

Resolved

OR



Restaurant Closed

Rejected

Creating a Promise

Syntax:

```
const promise = new Promise(  
  (resolve, reject) => {  
  
  }  
);
```

Example 1

```
const promise =  
new Promise((resolve, reject) => {  
  
  resolve("Success");  
  
});
```

Output

```
console.log(promise);
```

Output:

```
Promise { fulfilled: "Success" }
```

Understanding resolve()

```
resolve("Data Found");
```

means:

```
Operation Successful
```

Understanding reject()

```
reject("Error Occurred");
```

means:

```
Operation Failed
```

Example 2

```
const promise =  
new Promise((resolve, reject) => {  
  
    const success = true;  
  
    if(success) {
```

```
        resolve("Payment Successful");
    } else {
        reject("Payment Failed");
    }
});
```

Consuming a Promise

To receive the result:

```
.then()  
.catch()
```

Example

```
const promise =  
new Promise((resolve) => {  
    resolve("Data Loaded");  
});  
  
promise.then((data) => {  
    console.log(data);  
});
```

Output:

```
Data Loaded
```


How then() Works

```
promise.then(callback);
```

When Promise becomes:

Resolved

the callback executes.

Example

```
const promise =  
new Promise((resolve) => {  
    resolve("User Found");  
});  
  
promise.then((result) => {  
    console.log(result);  
});
```

Output:

User Found

Handling Errors with catch()

```
const promise =  
new Promise((resolve, reject) => {
```

```
        reject("Invalid Password");

    });

    promise.catch((error) => {

        console.log(error);

    });
```

Output:

```
Invalid Password
```

Real Industry Example

Login API

```
const login =
new Promise((resolve, reject) => {

    const password = "12345";

    if(password === "12345") {

        resolve("Login Successful");

    } else {

        reject("Login Failed");

    }

});
```

Consume

```
login
  .then(data => {

    console.log(data);

  })
  .catch(error => {

    console.log(error);

  });
```

Promise Chaining

One of the biggest benefits.

Example

```
new Promise((resolve) => {

  resolve(10);

})
.then(num => {

  return num * 2;

})
.then(num => {

  return num * 3;

})
.then(result => {

  console.log(result);
```

```
});
```

Output:

```
60
```

Visualization

```
10  
↓  
20  
↓  
60  
↓  
Output
```

Industry Example

E-Commerce Flow

```
loginUser()  
  .then(getProfile)  
  .then(getOrders)  
  .then(processPayment)  
  .then(showInvoice)  
  .catch(handleError);
```

Much cleaner than callback hell.

finally()

Runs regardless of success or failure.

Example

```
new Promise((resolve) => {  
  
    resolve("Success");  
  
})  
    .finally(() => {  
  
        console.log("Request Completed");  
  
    });
```

Output:

```
Request Completed
```

Why finally()?

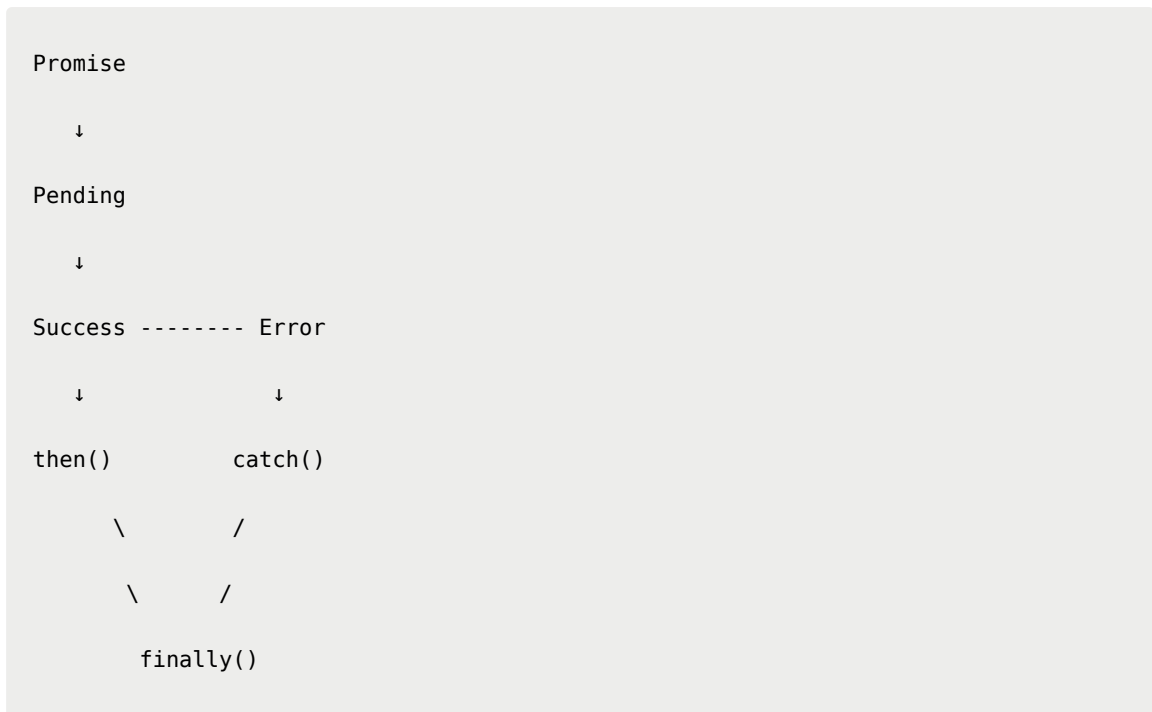
Real-world example:

```
Show Loader  
↓  
API Call  
↓  
Hide Loader
```

Whether API succeeds or fails:

```
finally(() => {  
  
    hideLoader();  
  
});
```

Promise Flow Diagram



Example: Fake API Call

```
function fetchUsers() {  
  
  return new Promise((resolve) => {  
  
    setTimeout(() => {  
  
      resolve([  
        "Rahul",  
        "Amit",  
        "Anand"  
      ]);  
  
    }, 2000);  
  
  });  
  
}
```

Usage

```
fetchUsers()  
  .then(users => {  
  
    console.log(users);  
  
  });
```

Output after 2 sec:

```
[  
  "Rahul",  
  "Amit",  
  "Anand"  
]
```

Promise APIs

JavaScript provides powerful APIs for handling multiple promises.

1. Promise.all()

Waits for:

```
ALL Promises
```

to succeed.

If one fails:

```
Entire Promise Fails
```

Example

```
const p1 =  
Promise.resolve("User");
```

```
const p2 =  
Promise.resolve("Orders");  
  
const p3 =  
Promise.resolve("Payment");  
  
Promise.all([p1,p2,p3])  
.then(result => {  
  
    console.log(result);  
  
});
```

Output:

```
[  
  "User",  
  "Orders",  
  "Payment"  
]
```

Real Industry Example

Dashboard Loading

```
User API  
Orders API  
Notifications API
```

Load all together.

```
Promise.all([  
  getUser(),  
  getOrders(),  
  getNotifications()  
]);
```


Important Interview Question

```
Promise.all([
  Promise.resolve(1),
  Promise.reject("Error"),
  Promise.resolve(3)
]);
```

Output:

```
Rejected Immediately
```

Because one promise failed.

2. Promise.race()

Returns:

```
First Settled Promise
```

(success OR failure)

Example

```
const p1 =
new Promise(resolve =>
setTimeout(() => resolve("Fast"),1000));

const p2 =
new Promise(resolve =>
setTimeout(() => resolve("Slow"),3000));

Promise.race([p1,p2])
.then(console.log);
```

Output:

Fast

Real World Example

API Timeout

```
Promise.race([
  fetchData(),
  timeoutPromise()
]);
```

Whichever finishes first wins.

3. Promise.allSettled()

Waits for:

ALL Promises

regardless of success or failure.

Example

```
Promise.allSettled([
  Promise.resolve("User"),
  Promise.reject("Error")
])
.then(console.log);
```

Output:

```
[
  {
    status: "fulfilled",
    value: "User"
  },

```

```
{  
  status: "rejected",  
  reason: "Error"  
}  
]
```

Real Industry Example

Loading Dashboard Widgets

```
Weather Widget  
News Widget  
Stocks Widget
```

If one widget fails:

```
Show remaining widgets
```

Use:

```
Promise.allSettled()
```

4. Promise.any()

Returns:

```
First Successful Promise
```

Ignores failures.

Example

```
Promise.any([  
  Promise.reject("Error"),  
  Promise.resolve("Success"),  
  Promise.resolve("Another Success")  
])
```

```
] )  
.then(console.log);
```

Output:

```
Success
```

Real World Example

Multiple Servers

```
Server India  
Server US  
Server Europe
```

Whichever responds successfully first:

```
Promise.any([  
  indiaServer(),  
  usServer(),  
  europeServer()  
]);
```

Promise APIs Comparison

API	Success Condition	Failure Condition
Promise.all()	All Success	One Failure
Promise.race()	First Settled	First Rejected
Promise.allSettled()	Always Completes	Never Rejects
Promise.any()	First Success	All Fail

Most Asked Interview Question

Predict Output

```
console.log("Start");

Promise.resolve()
  .then(() => {

    console.log("Promise");

  });

console.log("End");
```

Output:

```
Start
End
Promise
```

Why?

Because:

```
Promise Callbacks
go to
Microtask Queue
```

▼ Async & Await, and Error Handling

JavaScript Async & Await and Error Handling

Async/Await is built on top of Promises and is the modern way of handling asynchronous operations in JavaScript.

Before learning Async/Await, students must understand:

- ✓ Asynchronous JavaScript
- ✓ Event Loop
- ✓ Promises
- ✓ Promise Chaining

Because Async/Await internally uses Promises.

Why Was Async/Await Introduced?

Let's first understand the problem.

Promise Chaining Example

Suppose we need to:

1. Login User
2. Fetch Profile
3. Fetch Orders
4. Process Payment

Using Promises:

```
loginUser()  
  .then(user => {  
    return getProfile(user);  
  })  
  .then(profile => {  
    return getOrders(profile);  
  })  
  .then(orders => {  
    return processPayment(orders);  
  })  
  .then(result => {  
    console.log(result);  
  })  
  .catch(error => {  
    console.log(error);  
  });
```

Works perfectly.

But when applications become large:

```
10 APIs  
20 APIs
```

Code becomes difficult to read.

Real Life Analogy

Imagine reading instructions.

Promise Style

```
Go to Kitchen
  ↓
Then take Milk
  ↓
Then boil Milk
  ↓
Then make Tea
```

Async/Await Style

```
Go to Kitchen

Take Milk

Boil Milk

Make Tea
```

Much cleaner.

What is Async/Await?

Async/Await allows us to write asynchronous code that looks like synchronous code.

Two Keywords

```
async
await
```

async Keyword

The `async` keyword makes a function asynchronous.

Example:

```
async function greet() {  
  
}
```

Important Rule

An async function always returns a Promise.

Example

```
async function greet() {  
    return "Hello";  
}  
  
console.log(greet());
```

Output:

```
Promise { "Hello" }
```

Notice:

JavaScript automatically wraps
the value inside a Promise.

Visualization

```
async function greet() {  
    return "Hello";  
}
```


Internally behaves like:

```
function greet() {  
  
    return Promise.resolve(  
        "Hello"  
    );  
}
```

Example

```
async function greet() {  
  
    return "Welcome Students";  
}  
  
greet()  
    .then(data => {  
  
        console.log(data);  
  
    });
```

Output:

```
Welcome Students
```

await Keyword

`await` pauses execution until a Promise is resolved.

Example

```
const promise =  
    Promise.resolve("Data Loaded");  
  
async function getData() {
```

```
const result =  
await promise;  
  
console.log(result);  
}  
  
getData();
```

Output:

Data Loaded

Real Life Analogy

Imagine ordering coffee.

Without await:

Order Coffee
Continue Running

With await:

Order Coffee
Wait For Coffee
Drink Coffee
Continue

Example Using setTimeout

Create Promise

```
function fetchData() {  
  
    return new Promise(  
        (resolve) => {
```

```

        setTimeout(() => {

            resolve("Data Loaded");

        }, 3000);

    }

);

}

```

Using Async/Await

```

async function getData() {

    console.log("Loading...");

    const result =
    await fetchData();

    console.log(result);
}

getData();

```

Output:

```

Loading...

(wait 3 seconds)

Data Loaded

```

Behind The Scenes

```

const result =
await fetchData();

```

means:

Pause this function
until Promise resolves

Not:

Pause entire JavaScript

This is very important.

Sequential Execution

Example

```
function getUser() {  
    return Promise.resolve(  
        "User"  
    );  
}  
  
function getOrders() {  
    return Promise.resolve(  
        "Orders"  
    );  
}
```

Using await

```
async function loadData() {  
  
    const user =  
    await getUser();  
  
    console.log(user);  
  
    const orders =
```

```
    await getOrders();

    console.log(orders);
}

loadData();
```

Output:

```
User
Orders
```

Visualization

```
getUser()
Wait
↓
User
↓
getOrders()
Wait
↓
Orders
```

Industry Example

E-Commerce Website

```
Login User
↓
Fetch Profile
↓
Fetch Orders
```

↓
Show Dashboard

Code

```
async function dashboard() {  
  
  const user =  
    await loginUser();  
  
  const profile =  
    await getProfile();  
  
  const orders =  
    await getOrders();  
  
  console.log(  
    user,  
    profile,  
    orders  
  );  
}
```

Looks synchronous.

Actually asynchronous.

Parallel Execution

Many students don't know this optimization.

Wrong Approach

```
async function loadData() {  
  
  const users =  
    await getUsers();  
  
  const products =
```

```
    await getProducts();

    const orders =
    await getOrders();

}
```

Execution:

```
Users
Wait
↓
Products
Wait
↓
Orders
Wait
```

Total time:

```
3 + 3 + 3 = 9 sec
```

Better Approach

```
async function loadData() {

    const [
        users,
        products,
        orders
    ] = await Promise.all([
        getUsers(),
        getProducts(),
        getOrders()
    ]);

}
```

Execution

Users
Products
Orders

Run simultaneously.

Total:

3 sec

instead of

9 sec

Error Handling

Now comes the most important part.

Problem

```
async function getData() {  
  
    const result =  
    await fetchData();  
  
    console.log(result);  
}
```

What if:

API Fails?
Network Error?
Server Down?

Application crashes.

Solution

Use:

```
try  
catch
```

What is try-catch?

Used to handle runtime errors.

Syntax

```
try {  
  
}  
catch(error) {  
  
}
```

Example

```
try {  
  
    console.log(a);  
  
}  
catch(error) {  
  
    console.log(error);  
  
}
```

Output:

```
ReferenceError
```

Program continues running.

Real Life Analogy

Imagine driving a car.

Normal Road



Suddenly:

Road Block

Without Error Handling:

Crash

With Error Handling:

Take Alternative Route

Promise Rejection Example

```
function fetchData() {  
  return new Promise(  
    (resolve, reject) => {  
      reject(  
        "Server Error"  
      );  
    }  
  );  
}
```

Without Error Handling

```
async function getData() {  
  
    const result =  
    await fetchData();  
  
    console.log(result);  
}  
  
getData();
```

Error occurs.

Proper Error Handling

```
async function getData() {  
  
    try {  
  
        const result =  
        await fetchData();  
  
        console.log(result);  
  
    }  
    catch(error) {  
  
        console.log(  
            error  
        );  
    }  
  
}
```

```
getData();
```

Output:

Server Error

Industry Example

Fetching Products

```
async function getProducts() {  
  
    try {  
  
        const products =  
        await fetchProducts();  
  
        console.log(products);  
  
    }  
    catch(error) {  
  
        console.log(  
            "Unable To Load Products"  
        );  
  
    }  
  
}
```

finally Block

Runs always.

Example

```
try {  
  
    console.log("Success");  
  
}
```

```
catch(error) {  
  
    console.log(error);  
  
}  
finally {  
  
    console.log(  
        "Always Runs"  
    );  
  
}
```

Output:

```
Success  
Always Runs
```

Real Industry Example

Loading Spinner

```
showLoader();  
  
try {  
  
    const data =  
    await fetchProducts();  
  
}  
catch(error) {  
  
    console.log(error);  
  
}  
finally {  
  
    hideLoader();  
  
}
```

```
}
```

Whether API succeeds or fails:

Loader Removed

throw Keyword

Used to create custom errors.

Example

```
function validateAge(age) {  
  
    if(age < 18) {  
  
        throw new Error(  
            "Age Must Be 18+"  
        );  
    }  
  
    return "Valid";  
}
```

Usage

```
try {  
  
    console.log(  
        validateAge(15)  
    );  
}  
catch(error) {  
  
    console.log(  
        error.message  
    );  
}
```

```
);  
  
}
```

Output:

```
Age Must Be 18+
```

Industry Example

Registration Form

```
function register(age) {  
  
    if(age < 18) {  
  
        throw new Error(  
            "Not Eligible"  
        );  
    }  
  
    return "Registered";  
}
```

Custom Error Example

```
async function login() {  
  
    const password =  
        "123";  
  
    if(password !== "12345") {  
  
        throw new Error(  
            "Invalid Password"  
        );  
    }  
}
```

```
    return "Login Success";  
}
```

Complete Async/Await Example

```
function fetchUser() {  
  
    return new Promise(  
        (resolve) => {  
  
            setTimeout(() => {  
  
                resolve({  
                    id:1,  
                    name:"Rahul"  
                });  
  
            },2000);  
  
        }  
    );  
}
```

Consume

```
async function getUser() {  
  
    try {  
  
        console.log(  
            "Loading..."  
        );  
  
        const user =  
            await fetchUser();  
  
    }  
}
```



```

        console.log(user);
    }
    catch(error) {

        console.log(error);

    }
    finally {

        console.log(
            "Request Completed"
        );

    }

}

getUser();

```

Output:

```

Loading...

(wait 2 sec)

{
  id:1,
  name:"Rahul"
}

Request Completed

```

Async/Await vs Promise

Promise Style

```

fetchUsers()
  .then(data => {

```

```
        console.log(data);  
  
    })  
    .catch(error => {  
  
        console.log(error);  
  
    });
```

Async/Await Style

```
try {  
  
    const data =  
    await fetchUsers();  
  
    console.log(data);  
  
}  
catch(error) {  
  
    console.log(error);  
  
}
```

Cleaner and easier to read.

▼ Fetch API

JavaScript Fetch API

The **Fetch API** is one of the most important APIs in modern JavaScript. It is used to communicate with servers and APIs.

Almost every modern application uses Fetch API:

- ✓ E-Commerce Applications
- ✓ Banking Applications
- ✓ Social Media Applications

- ✓ Weather Applications
 - ✓ Food Delivery Applications
 - ✓ React Applications
 - ✓ MERN Stack Applications
-

What is an API?

Before learning Fetch API, students must understand:

API = Application Programming Interface

Real Life Analogy

Imagine you go to a restaurant.

You don't directly enter the kitchen.

Instead:

```
Customer
  ↓
Waiter
  ↓
Kitchen
  ↓
Waiter
  ↓
Customer
```

The waiter acts as a bridge.

Similarly:

```
Frontend
  ↓
API
  ↓
Backend
  ↓
Database
```

The API acts as a bridge between the frontend and backend.

Real Industry Example

Amazon Application

```
Frontend (React)
  ↓
API
  ↓
Backend (Node.js)
  ↓
Database
```

When you search for a product:

Laptop

The frontend asks the API:

Give me all laptops

The API returns:

```
[
  {
    "id":1,
    "name":"HP Laptop",
    "price":50000
  }
]
```

What is Fetch API?

Definition

Fetch API is a modern JavaScript API used to make HTTP requests to servers.

Simple Definition

Fetch API allows JavaScript to:

Send Requests
Receive Responses
Communicate with APIs

Syntax

```
fetch(url)
```

Example:

```
fetch(  
  "https://jsonplaceholder.typicode.com/users"  
);
```

Important Interview Question

Does fetch return data directly?

Answer:

NO

Fetch returns:

Promise

Example

```
const result = fetch(  
  "https://jsonplaceholder.typicode.com/users"  
);  
  
console.log(result);
```

Output:

```
Promise { <pending> }
```

Why Promise?

Because:

Network Requests Take Time

For example:

India → USA Server

might take:

100ms
500ms
1000ms

JavaScript cannot block execution.

Therefore:

```
fetch()  
returns a Promise
```

Understanding HTTP Requests

When using Fetch API, we send HTTP requests.

Common HTTP Methods

Method	Purpose
GET	Retrieve Data
POST	Create Data
PUT	Update Entire Data

Method	Purpose
PATCH	Update Partial Data
DELETE	Delete Data

Real Life Example

Student Management System

GET
→ Fetch Students

POST
→ Add Student

PUT
→ Update Student

DELETE
→ Remove Student

GET Request

Used to retrieve data.

Example

```
fetch(  
  "https://jsonplaceholder.typicode.com/users"  
)  
  .then(response => {  
  
    console.log(response);  
  
  });
```

What Does Fetch Return?

Fetch returns a Response Object.

```
Response {  
  status: 200,  
  ok: true,  
  headers: {},  
  body: {}  
}
```

Important Question

Where is the actual data?

Not here:

```
response
```

Because:

```
Response Object  
≠  
Actual Data
```

Convert Response to JSON

```
fetch(  
  "https://jsonplaceholder.typicode.com/users"  
)  
  .then(response => {  
    return response.json();  
  })  
  .then(data => {  
    console.log(data);  
  })
```



```
});
```

Why response.json()?

Because APIs usually return:

```
{  
  "id": 1,  
  "name": "Rahul"  
}
```

JSON format.

Visualization

```
Server  
↓  
JSON Data  
↓  
response.json()  
↓  
JavaScript Object
```

Full GET Example

```
fetch(  
  "https://jsonplaceholder.typicode.com/users"  
)  
  .then(response => {  
    return response.json();  
  })
```

```
.then(users => {  
  
    console.log(users);  
  
})  
.catch(error => {  
  
    console.log(error);  
  
});
```

Using Async/Await

Modern approach.

```
async function getUsers() {  
  
    try {  
  
        const response =  
        await fetch(  
            "https://jsonplaceholder.typicode.com/users"  
        );  
  
        const users =  
        await response.json();  
  
        console.log(users);  
  
    }  
    catch(error) {  
  
        console.log(error);  
  
    }  
}
```

```
getUsers();
```

Understanding Response Object

Example:

```
const response =  
await fetch(url);
```

Response contains:

```
response.status  
response.ok  
response.headers  
response.body
```

response.status

```
console.log(  
  response.status  
);
```

Output:

```
200
```

Common Status Codes

Status	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden

Status	Meaning
404	Not Found
500	Server Error

response.ok

```
console.log(  
  response.ok  
);
```

Output:

```
true
```

Used to check:

```
Request Successful?
```

Example

```
if(!response.ok) {  
  
  throw new Error(  
    "Request Failed"  
  );  
}
```

POST Request

Used to create new data.

Real Life Example

Add Student

Student Name
Student Age

Send to Server.

Example

```
fetch(  
  "https://jsonplaceholder.typicode.com/posts",  
  {  
    method: "POST",  
  
    headers: {  
      "Content-Type":  
        "application/json"  
    },  
  
    body: JSON.stringify({  
      title: "JavaScript",  
      body: "Learning Fetch API"  
    })  
  })  
);
```

Understanding Headers

Headers provide additional information.

Most common:

```
headers: {  
  "Content-Type":  
    "application/json"  
}
```

Meaning:

I am sending JSON Data

Understanding Body

Body contains data sent to server.

Example

```
body: JSON.stringify({  
  
  title: "JavaScript",  
  
  body: "Learning Fetch"  
  
})
```

Full POST Example

```
async function createPost() {  
  
  const response =  
  await fetch(  
    "https://jsonplaceholder.typicode.com/posts",  
    {  
  
      method: "POST",  
  
      headers: {  
        "Content-Type":  
        "application/json"  
      },  
  
      body: JSON.stringify({  
  
        title: "JavaScript",  
  
        body: "Fetch API"  
  
      })  
    }  
  )  
}
```

```
});  
  
const data =  
await response.json();  
  
console.log(data);  
}
```

PUT Request

Used to update existing data completely.

Example

```
fetch(  
  "https://jsonplaceholder.typicode.com/posts/1",  
  {  
    method: "PUT",  
  
    headers: {  
      "Content-Type":  
        "application/json"  
    },  
  
    body: JSON.stringify({  
  
      id: 1,  
  
      title: "Updated",  
  
      body: "New Content"  
    })  
  });
```

DELETE Request

Used to remove data.

Example

```
fetch(  
  "https://jsonplaceholder.typicode.com/posts/1",  
  {  
    method: "DELETE"  
  });
```

Industry Example

E-Commerce

GET
→ Products

POST
→ Add Product

PUT
→ Update Product

DELETE
→ Delete Product

Handling Errors Properly

Many beginners do:

```
const response =  
await fetch(url);
```

and stop.

Wrong.

Always check:


```
if(!response.ok) {  
  
    throw new Error(  
        "Request Failed"  
    );  
}
```

Complete Example

```
async function getUsers() {  
  
    try {  
  
        const response =  
            await fetch(url);  
  
        if(!response.ok) {  
  
            throw new Error(  
                "Failed To Fetch Data"  
            );  
        }  
  
        const users =  
            await response.json();  
  
        console.log(users);  
  
    }  
    catch(error) {  
  
        console.log(  
            error.message  
        );  
    }  
  
}
```

Real Industry Example

Weather Application

```
const response =  
await fetch(weatherAPI);
```

↓

```
const data =  
await response.json();
```

↓

```
console.log(  
  data.temperature  
);
```

Complete Flow

```
User Clicks Button  
  ↓  
fetch()  
  ↓  
Request Sent  
  ↓  
Server Receives Request  
  ↓  
Database Query  
  ↓  
Response Sent  
  ↓  
response.json()  
  ↓  
JavaScript Object  
  ↓  
Display Data
```

Fetch vs XMLHttpRequest

Before Fetch API:

```
const xhr =  
new XMLHttpRequest();
```

Very complex.

Now:

```
const response =  
await fetch(url);
```

Much simpler.

▼ Modules

JavaScript Modules

Modules are one of the most important ES6 features. They help us split large applications into smaller, reusable, and maintainable files.

Without Modules:

```
1 File  
  ↓  
5000 Lines  
  ↓  
Difficult to Maintain
```

With Modules:

```
App  
├──  
├── user.js  
├── product.js  
├── auth.js  
├── payment.js  
└── app.js
```

Much cleaner and scalable.

Why Do We Need Modules?

Imagine you're building an E-Commerce Application.

Features:

```
Login  
Products  
Cart  
Payments  
Orders  
Users
```

Without Modules:

```
// app.js  
  
function login(){}  
  
function logout(){}  
  
function addToCart(){}  
  
function removeFromCart(){}  
  
function processPayment(){}  
  
function placeOrder(){}  
  
// 5000+ lines
```

Problem:

```
Difficult to Read  
Difficult to Debug  
Difficult to Reuse  
Difficult to Scale
```

Real Life Analogy

Imagine a university.

Without departments:

```
Students
Teachers
Accounts
Library
Placement
Exams
```

all in one room.

Chaos.

With departments:

```
Accounts Department
Library Department
Placement Department
Exam Department
```

Everything organized.

Modules do exactly the same thing.

What is a Module?

Definition

A Module is a JavaScript file that contains reusable code and can export or import functionality from other files.

Benefits of Modules

1. Code Reusability

```
export function add() {}
```

Use anywhere.

2. Code Organization

Separate responsibilities.

3. Maintainability

Easy to update.

4. Encapsulation

Variables remain private unless exported.

Before ES6 Modules

Old JavaScript:

```
<script src="math.js"></script>
<script src="user.js"></script>
<script src="app.js"></script>
```

Problems:

```
Global Scope Pollution
Variable Conflicts
Dependency Issues
```

Example

math.js

```
var count = 10;
```

user.js

```
var count = 20;
```

Conflict occurs.

ES6 Modules Solution

Each file gets its own scope.

```
const count = 10;
```

inside:

```
math.js
```

won't affect:

```
user.js
```

Using Modules in Vanilla JavaScript

Project Structure

```
project/  
|  
├─ index.html  
├─ app.js  
├─ math.js  
└─ user.js
```

Step 1: Export Function

math.js

```
export function add(a, b) {  
  return a + b;  
}  
  
export function multiply(a, b) {  
  return a * b;  
}
```

Step 2: Import Function

app.js

```
import {  
  add,  
  multiply  
}  
from "./math.js";  
  
console.log(add(10, 20));  
  
console.log(multiply(5, 5));
```

Output:

```
30  
25
```

Step 3: HTML Setup

index.html

```
<script  
  type="module"  
  src="./app.js">  
</script>
```

Very Important:

```
type="module"
```

Without it:

```
Import Error
```

Named Export

Most common.

math.js


```
export const PI = 3.14;

export function add(a,b){
  return a+b;
}

export function subtract(a,b){
  return a-b;
}
```

Import

```
import {
  PI,
  add,
  subtract
}
from "./math.js";
```

Why Curly Braces?

Because:

Named Export

requires:

```
{}
```

Multiple Named Exports

math.js

```
function add(a,b){
  return a+b;
}
```

```
function multiply(a,b){
    return a*b;
}

export {
    add,
    multiply
};
```

Import

```
import {
    add,
    multiply
}
from "./math.js";
```

Alias Import

Suppose:

math.js

```
export function add(a,b){
    return a+b;
}
```

Import

```
import {
    add as sum
}
from "./math.js";

console.log(sum(5,5));
```

Output:

Default Export

Only one default export per file.

math.js

```
export default function add(a,b){  
  return a+b;  
}
```

Import

```
import add  
from "./math.js";
```

Notice:

No Curly Braces

Another Example

user.js

```
export default class User {  
  
  constructor(name){  
  
    this.name = name;  
  
  }  
  
}
```

Import

```
import User
from './user.js';
```

Named Export vs Default Export

Feature	Named	Default
Multiple Allowed	✓	✗
Curly Braces Needed	✓	✗
Rename While Import	using as	Any Name

Example

Export

```
export default function greet(){}
```

Import

```
import abc from './file.js';
```

Works.

Because default import can have any name.

Mixed Export

A file can contain both.

math.js

```
export const PI = 3.14;

export function add(a,b){
  return a+b;
}

export default function multiply(a,b){
```

```
    return a*b;
}
```

Import

```
import multiply,
{
    PI,
    add
}
from "./math.js";
```

Import Everything

math.js

```
export const PI = 3.14;

export function add(){}
export function subtract(){}

```

Import

```
import * as MathUtils
from "./math.js";

console.log(
    MathUtils.PI
);
```

Output

```
3.14
```

Dynamic Import

Load module when needed.

Example

```
const module =  
await import(  
  './math.js'  
);  
  
console.log(  
  module.add(5,5)  
);
```

Useful for:

```
Performance Optimization  
Lazy Loading
```

Module Scope

Example

math.js

```
const secret =  
  "Admin Password";
```

app.js

```
console.log(secret);
```

Output:

```
ReferenceError
```

Why?

Because:

Modules have their own scope.

Real Industry Example (Vanilla JS)

Project Structure

Student Management

```
|  
├─ index.html  
├─ app.js  
├─ student.js  
├─ validation.js  
└─ api.js
```

student.js

```
export class Student {  
  
  constructor(  
    name,  
    age  
  ){  
    this.name = name;  
    this.age = age;  
  }  
}
```

validation.js

```
export function validateAge(  
  age  
) {  
  return age >= 18;  
}
```

app.js

```
import {
  Student
}
from "./student.js";

import {
  validateAge
}
from "./validation.js";
```

Modules in React

React uses modules everywhere.

Project Structure

```
src
├── App.jsx
├── components
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── Navbar.jsx
└── services
    └── api.js
```

Component Export

Header.jsx

```
function Header(){

  return (
    <h1>
      Header
    </h1>
  )
}
```



```
    );  
  }  
  
  export default Header;
```

Import

App.jsx

```
import Header  
from "../components/Header";  
  
function App(){  
  
  return (  
    <>  
      <Header />  
    </>  
  );  
}
```

Named Export in React

Button.jsx

```
export function PrimaryButton(){  
  
  return (  
    <button>  
      Save  
    </button>  
  );  
}  
  
export function DangerButton(){  
  
  return (  
    <button>
```

```
        Delete
      </button>
    );
  }
}
```

Import

```
import {
  PrimaryButton,
  DangerButton
}
from "./Button";
```

Exporting API Services

api.js

```
export async function
getUsers(){

  const response =
  await fetch(url);

  return response.json();
}
```

Import

```
import {
  getUsers
}
from "./api";
```

Barrel Exports

Large React projects use:

```
index.js
```

to re-export everything.

components/index.js

```
export {  
  default as Header  
}  
from "./Header";  
  
export {  
  default as Footer  
}  
from "./Footer";
```

Import

```
import {  
  Header,  
  Footer  
}  
from "./components";
```

Cleaner imports.

Common Module Errors

Error 1

```
Cannot use import statement  
outside a module
```

Solution:

```
<script type="module">
```

Error 2

Wrong path

```
import add from "math.js";
```

Wrong.

Use:

```
import add from "./math.js";
```

Error 3

Default vs Named Confusion

Export:

```
export function add(){}
```

Wrong Import:

```
import add from "./math.js";
```

Correct:

```
import { add }  
from "./math.js";
```

▼ CDN

CDN (Content Delivery Network)

What is a CDN?

CDN (Content Delivery Network) is a network of servers located in different parts of the world that store copies of files such as:

- JavaScript files
- CSS files
- Images
- Videos
- Fonts

When a user requests a file, the CDN provides it from the nearest server instead of the original server.

Why Do We Need a CDN?

Without CDN:

```
Browser -----> Original Server
```

Every user must download files from the same server.

Problems:

- Slow loading
- High server load
- Poor user experience

With CDN:

```
          CDN Server 1
          |
Browser -----> Nearest CDN Server
          |
          CDN Server 2
          |
          CDN Server 3
```

Benefits:

- Faster loading
- Better performance

- Reduced server load
 - High availability
 - Browser caching support
-

CDN in React

Before modern tools like Vite, Create React App, or Webpack, React could be used directly in HTML using CDN links.

Example:

```
<div id="root"></div>

<script crossorigin src="https://unpkg.com/react@18/umd/
react.development.js"></script>

<script crossorigin src="https://unpkg.com/react-dom@18/
umd/react-dom.development.js"></script>

<script src="https://unpkg.com/babel-standalone@6/babel.
min.js"></script>
```

Understanding Each CDN

1. React CDN

```
<script crossorigin src="https://unpkg.com/react@18/umd/
react.development.js"></script>
```

Purpose

Loads the React Library.

What React Does

React helps us create UI elements.

Example:

```
React.createElement(  
  "h1",  
  {},  
  "Hello World"  
);
```

Without React:

```
React.createElement()
```

produces:

```
ReferenceError: React is not defined
```

2. ReactDOM CDN

```
<script crossorigin src="https://unpkg.com/react-dom@18/  
umd/react-dom.development.js"></script>
```

Purpose

Connects React with the Browser DOM.

What ReactDOM Does

React can create elements.

ReactDOM displays those elements inside HTML.

Example:

```
const root = ReactDOM.createRoot(  
  document.getElementById("root")  
);
```

Without ReactDOM:

```
React cannot render anything on the webpage.
```

3. Babel CDN

```
<script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
```

Purpose

Converts JSX into JavaScript.

What is JSX?

JSX stands for:

JavaScript XML

Example:

```
const element = <h1>Hello React</h1>;
```

Browsers do not understand JSX.

Babel Conversion

Babel converts:

```
const element = <h1>Hello React</h1>;
```

into:

```
const element = React.createElement(  
  "h1",  
  null,  
  "Hello React"  
);
```

Now browsers can execute the code.

Understanding Root Element


```
<div id="root"></div>
```

This is the container where React displays UI.

Example:

```
const root = ReactDOM.createRoot(  
  document.getElementById("root")  
);
```

React gets access to:

```
<div id="root"></div>
```

and renders content inside it.

Complete React Flow

Step 1

HTML loads

```
<div id="root"></div>
```

Output:

```
Empty Page
```

Step 2

React CDN loads

```
<script src="react.development.js"></script>
```

React becomes available.

Step 3

ReactDOM CDN loads

```
<script src="react-dom.development.js"></script>
```

React can now interact with the DOM.

Step 4

Babel CDN loads

```
<script src="babel.min.js"></script>
```

JSX can now be converted to JavaScript.

Step 5

React Code Executes

```
<script type="text/babel">

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <h1>Hello React 🚀</h1>
);

</script>
```

What Happens Internally?

JSX

```
<h1>Hello React 🚀</h1>
```

↓

Babel Converts

```
React.createElement(  
  "h1",  
  null,  
  "Hello React 🚀"  
);
```

↓

React Creates Virtual DOM Element

↓

ReactDOM Renders to Browser DOM

↓

Final Output

Hello React 🚀

Advantages of CDN

1. Fast Loading

Nearest server delivers files.

2. Easy Setup

No installation required.

3. Browser Caching

If React is already downloaded, browser may reuse it.

4. Good for Learning

Perfect for beginners.

5. Less Server Load

Original server handles fewer requests.

Disadvantages of CDN

1. Internet Required

No internet = CDN files cannot load.

2. Less Suitable for Large Projects

Modern React applications use:

- Vite
- Webpack
- Parcel
- Next.js

instead of CDN.

3. Slower Than Build Tools

Large applications need optimization provided by build tools.

Nodejs Backend

▼ Nodejs Inbuilt Modules

Node.js Important Inbuilt Modules

What are Inbuilt Modules?

Node.js provides several built-in modules that can be used directly without installing any package.

Example:

```
const fs = require("fs");
```

No need to install anything.

1. FS Module (File System)

The `fs` module is used to create, read, update, delete, and manage files.

```
const fs = require("fs");
```

Write File

Synchronous

```
const fs = require("fs");

fs.writeFileSync("data.txt", "Hello Students");

console.log("File Created");
```

Output:

```
File Created
```

File:

```
Hello Students
```

Asynchronous

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello Node", (err) => {
  if (err) {
    console.log(err);
    return;
  }
})
```

```
    console.log("File Created");  
  });
```

Read File

Synchronous

```
const data = fs.readFileSync("data.txt", "utf8");  
  
console.log(data);
```

Output:

```
Hello Node
```

Asynchronous

```
fs.readFile("data.txt", "utf8", (err, data) => {  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
  console.log(data);  
});
```

Append Data

```
fs.appendFileSync("data.txt", "\nWelcome To MERN");
```

File:

```
Hello Node
```

```
Welcome To MERN
```

Delete File

```
fs.unlinkSync("data.txt");
```

Rename File

```
fs.renameSync("old.txt", "new.txt");
```

Check File Exists

```
console.log(fs.existsSync("data.txt"));
```

Output:

```
true
```

Real Life Use Cases

- Generate reports
- Store logs
- Upload files
- Read configuration files

2. PATH Module

The `path` module helps handle file and folder paths.

```
const path = require("path");
```

Join Paths

```
const result = path.join("uploads", "images", "photo.jpg");

console.log(result);
```

Output:

```
uploads/images/photo.jpg
```

Get Filename

```
console.log(path.basename("/users/admin/photo.jpg"));
```

Output:

```
photo.jpg
```

Get Extension

```
console.log(path.extname("resume.pdf"));
```

Output:

```
.pdf
```

Directory Name

```
console.log(path.dirname("/users/admin/file.txt"));
```

Output:


```
/users/admin
```

Absolute Path

```
console.log(path.resolve("data.txt"));
```

Output:

```
/Users/anand/project/data.txt
```

Real Life Use Cases

- Upload folder management
- Dynamic file storage
- Static file serving

3. OS Module

Provides operating system information.

```
const os = require("os");
```

Platform

```
console.log(os.platform());
```

Output:

```
darwin
```

or

```
win32
```

CPU Architecture

```
console.log(os.arch());
```

Output:

```
arm64
```

Total Memory

```
console.log(os.totalmem());
```

Output:

```
17179869184
```

(bytes)

Free Memory

```
console.log(os.freemem());
```

Host Name

```
console.log(os.hostname());
```

User Information

```
console.log(os.userInfo());
```

Output:

```
{  
  uid: 501,  
  username: 'anand'  
}
```

Real Life Use Cases

- Monitoring server health
 - System analytics dashboard
 - Cloud server monitoring
-

4. HTTP Module

Used to create a web server without Express.

```
const http = require("http");
```

Create Server

```
const http = require("http");  
  
const server = http.createServer((req, res) => {  
  
  res.write("Hello Node");  
  
  res.end();  
  
});  
  
server.listen(3000);
```

Visit:

```
http://localhost:3000
```

Output:

```
Hello Node
```

Route Handling

```
const server = http.createServer((req, res) => {  
  
  if (req.url === "/") {  
    res.end("Home");  
  }  
  
  else if (req.url === "/about") {  
    res.end("About");  
  }  
  
  else {  
    res.end("404");  
  }  
  
});
```

Returning JSON

```
const data = {  
  name: "Anand",  
  course: "MERN"  
};  
  
res.setHeader("Content-Type", "application/json");  
  
res.end(JSON.stringify(data));
```

Output:

```
{  
  "name": "Anand",  
  "course": "MERN"  
}
```

Real Life Use Cases

- API Development
- Microservices
- Backend Server

5. EVENTS Module

Node.js follows Event Driven Architecture.

```
const EventEmitter = require("events");
```

Create Event

```
const EventEmitter = require("events");  
  
const emitter = new EventEmitter();  
  
emitter.on("login", () => {  
  console.log("User Logged In");  
});  
  
emitter.emit("login");
```

Output:

```
User Logged In
```

Passing Data

```
emitter.on("purchase", (product) => {  
    console.log(product);  
});  
  
emitter.emit("purchase", "Laptop");
```

Output:

```
Laptop
```

Multiple Listeners

```
emitter.on("order", () => {  
    console.log("Email Sent");  
});  
  
emitter.on("order", () => {  
    console.log("SMS Sent");  
});  
  
emitter.emit("order");
```

Output:

```
Email Sent  
SMS Sent
```

Real Life Use Cases

- Notification Systems
- Order Processing
- Logging Systems

6. URL Module

Used to parse URLs.

```
const url = require("url");
```

Parse URL

```
const myUrl =  
url.parse("http://localhost:3000/products?id=101", true);  
  
console.log(myUrl.query);
```

Output:

```
{  
  id: '101'  
}
```

Get Pathname

```
console.log(myUrl.pathname);
```

Output:

```
/products
```

Real Life Use Cases

- Query Parameters
- URL Parsing
- Routing

7. CRYPTO Module

Used for encryption and hashing.

```
const crypto = require("crypto");
```

Create Hash

```
const crypto = require("crypto");

const hash = crypto
  .createHash("sha256")
  .update("password123")
  .digest("hex");

console.log(hash);
```

Output:

```
ef92b778...
```

Random Token

```
const token =
  crypto.randomBytes(16).toString("hex");

console.log(token);
```

Output:

```
5ab87f2e9d4...
```

Real Life Use Cases

- Password Hashing

- JWT Secret Generation
 - OTP Generation
-

8. STREAM Module

Used for handling large files efficiently.

Read Stream

```
const fs = require("fs");

const stream =
  fs.createReadStream("large.txt", "utf8");

stream.on("data", (chunk) => {
  console.log(chunk);
});
```

Write Stream

```
const writeStream =
  fs.createWriteStream("output.txt");

writeStream.write("Hello");
```

Real Life Use Cases

- Video Streaming
 - Large File Uploads
 - Log Processing
-

9. PROCESS Module

Provides information about the current Node process.

Command Line Arguments

```
console.log(process.argv);
```

Run:

```
node app.js hello
```

Output:

```
[  
  '/usr/bin/node',  
  '/app.js',  
  'hello'  
]
```

Environment Variables

```
console.log(process.env.PORT);
```

Current Working Directory

```
console.log(process.cwd());
```

10. TIMER MODULE

Node.js provides timers globally.

setTimeout

```
setTimeout(() => {  
  console.log("Hello");  
}, 3000);
```

setInterval

```
setInterval(() => {
  console.log("Running");
}, 1000);
```

clearInterval

```
const id = setInterval(() => {
  console.log("Running");
}, 1000);

clearInterval(id);
```

Most Important Modules for MERN Students

Module	Interview Importance	Project Usage
fs	★★★★★★	★★★★★★
path	★★★★★★	★★★★★★
http	★★★★★★	★★★★★★
events	★★★★★	★★★★★
os	★★★★	★★★★
crypto	★★★★★★	★★★★★★
process	★★★★★	★★★★★
url	★★★★★	★★★★
stream	★★★★★★	★★★★★★

▼ Http Module

Node.js HTTP Module

Chapter 1: What is HTTP Module?

The **HTTP Module** is a built-in Node.js module used to create web servers and handle HTTP requests and responses.

Before Express.js existed, developers used the HTTP module to build APIs and web applications.

```
const http = require("http");
```

Real Life Analogy

Imagine a restaurant.

Client (Customer)

Sends order

```
GET /students
```

Server (Kitchen)

Processes request

Response

Returns data

```
[
  {
    "id": 1,
    "name": "Anand"
  }
]
```

Creating First HTTP Server

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.end("Hello Students");
```

```
});  
  
server.listen(3000,()=>{  
  console.log("Server Started");  
});
```

Request Object (req)

Contains information sent by client.

```
req.url  
req.method  
req.headers
```

Example:

```
console.log(req.url);  
console.log(req.method);
```

Output:

```
/students  
GET
```

Response Object (res)

Used to send response.

```
res.write("Hello");  
res.end();
```

or

```
res.end("Hello");
```

Routing in HTTP Module

Example

```
const http = require("http");

const server = http.createServer((req, res) => {

  if (req.url === "/") {
    res.end("Home Page");
  }

  else if (req.url === "/students") {
    res.end("Students Page");
  }

  else {
    res.end("404 Not Found");
  }

});

server.listen(3000);
```

Returning JSON Data

```
const students = [
  {
    id: 1,
    name: "Anand"
  }
];

res.setHeader("Content-Type", "application/json");

res.end(JSON.stringify(students));
```

Output

```
[
  {
    "id": 1,
    "name": "Anand"
  }
]
```

Handling Request Method

```
if (req.method === "GET") {
}
```

```
if (req.method === "POST") {
}
```

```
if (req.method === "PUT") {
}
```

```
if (req.method === "DELETE") {
}
```

Reading Request Body

Very complicated in HTTP Module.

```
let body = "";

req.on("data", (chunk) => {
  body += chunk;
});

req.on("end", () => {

  const data = JSON.parse(body);
}
```

```
    console.log(data);  
  
  });
```

Client sends:

```
{  
  "name": "Anand"  
}
```

Output:

```
{  
  name: "Anand"  
}
```

Complete Student API Using HTTP Module

```
const http = require("http");  
  
const students = [];  
  
const server = http.createServer((req, res) => {  
  if (req.url === "/students" && req.method === "GET") {  
    res.setHeader("Content-Type", "application/json");  
    res.end(JSON.stringify(students));  
  }  
});  
  
server.listen(3000);
```

Works but becomes messy when project grows.

Problems With HTTP Module

Suppose project contains:

Students APIs

```
GET /students
POST /students
PUT /students
DELETE /students
```

Courses APIs

```
GET /courses
POST /courses
PUT /courses
DELETE /courses
```

Faculty APIs

```
GET /faculty
POST /faculty
PUT /faculty
DELETE /faculty
```

Code becomes:

```
if(req.url === "/students" && req.method==="GET"){

}
else if(req.url === "/students" && req.method==="POST"){

}
else if(req.url === "/courses" && req.method==="GET"){

}
else if(req.url === "/faculty" && req.method==="DELETE")
{
```

```
}
```

Hundreds of lines.

Hard to maintain.

Limitations of HTTP Module

1. Manual Routing

```
if(req.url === "/students")
```

Everything must be written manually.

2. Manual Body Parsing

```
req.on("data")
```

Very difficult.

3. No Middleware

Cannot easily execute logic before request.

Example:

```
Authentication  
Logging  
Validation
```

4. Poor Code Structure

Large projects become unmanageable.

5. Difficult Error Handling

Need custom logic everywhere.

6. No Route Grouping

Cannot separate:

```
student routes  
course routes  
faculty routes
```

Solution → Express.js

Express is a framework built on top of Node.js HTTP Module.

Express internally uses:

```
http.createServer()
```

but provides easier syntax.

What is Express?

Express is a minimal and flexible Node.js framework used to build APIs and Web Applications quickly.

Install:

```
npm init -y  
  
npm install express
```

First Express Server

```
const express = require("express");  
  
const app = express();  
  
app.listen(3000, ()=>{
```

```
    console.log("Server Started");  
  });
```

Route Handling

GET

```
app.get("/", (req, res) => {  
  res.send("Home");  
});
```

POST

```
app.post("/students", (req, res) => {  
  res.send("Student Created");  
});
```

PUT

```
app.put("/students/:id", (req, res) => {  
  res.send("Student Updated");  
});
```

DELETE

```
app.delete("/students/:id", (req, res) => {  
  res.send("Student Deleted");  
});
```

Middleware

One of the biggest features of Express.

Middleware executes before request reaches route.

```
app.use((req, res, next) => {  
  
  console.log("Request Received");  
  
  next();  
  
});
```

Middleware Flow

```
Request  
  ↓  
Middleware 1  
  ↓  
Middleware 2  
  ↓  
Route  
  ↓  
Response
```

Built-in Middleware

JSON Middleware

```
app.use(express.json());
```

Automatically converts:

```
{  
  "name": "Anand"  
}
```

into

```
req.body
```

Route Parameters

```
app.get("/student/:id", (req, res) => {  
    console.log(req.params.id);  
});
```

Request

```
/student/101
```

Output

```
101
```

Query Parameters

```
app.get("/search", (req, res) => {  
    console.log(req.query.name);  
});
```

Request

```
/search?name=anand
```

Output

```
anand
```

Express Router

Separate route files.

studentRoutes.js

```
const router = require("express").Router();

router.get("/", ()=>{});

router.post("/", ()=>{});

module.exports = router;
```

app.js

```
app.use("/students", studentRoutes);
```

Error Handling Middleware

```
app.use((err, req, res, next) => {

    res.status(500).json({
        message: err.message
    });

});
```

Advantages of Express

HTTP Module	Express
Manual Routing	Easy Routing
Manual Body Parsing	express.json()
No Middleware	Middleware Support
Large Code	Cleaner Code
Hard Maintenance	Easy Maintenance
Basic Features	Rich Features

Student Registration Application

Folder Structure

```
project
|
├── app.js
├── student.json
├── routes
│   └── studentRoutes.js
├── controllers
│   └── studentController.js
└── utils
    └── fileHelper.js
```

student.json

```
[]
```

fileHelper.js

```
const fs = require("fs");

function readStudents(){

    const data =
    fs.readFileSync("./student.json","utf8");

    return JSON.parse(data);
}

function saveStudents(students){

    fs.writeFileSync(
        "./student.json",
        JSON.stringify(students,null,2)
    );
}
```



```
module.exports = {  
  readStudents,  
  saveStudents  
};
```

studentController.js

```
const {  
  readStudents,  
  saveStudents  
}  
= require("../utils/fileHelper");
```

Create Student

```
exports.createStudent = (req, res) => {  
  
  const students = readStudents();  
  
  const student = {  
    id: Date.now(),  
    ...req.body  
  };  
  
  students.push(student);  
  
  saveStudents(students);  
  
  res.json(student);  
  
};
```

Get All Students

```
exports.getStudents = (req, res) => {  
  
  const students = readStudents();  
  
  res.json(students);  
  
};
```

Get Student By Id

```
exports.getStudentById = (req, res) => {  
  
  const students = readStudents();  
  
  const student =  
    students.find(  
      s => s.id === req.params.id  
    );  
  
  if (!student) {  
  
    return res.status(404)  
      .json({  
        message: "Not Found"  
      });  
  }  
  
  res.json(student);  
  
};
```

Update Student

```
exports.updateStudent = (req, res) => {  
  
  const students = readStudents();
```

```
const index =
students.findIndex(
  s=>s.id == req.params.id
);

if(index === -1){

  return res.status(404)
    .json({
      message:"Not Found"
    });
}

students[index]={
  ...students[index],
  ...req.body
};

saveStudents(students);

res.json(students[index]);

};
```

Delete Student

```
exports.deleteStudent=(req,res)=>{

  let students = readStudents();

  students =
students.filter(
  s=>s.id != req.params.id
);

saveStudents(students);
```

```
    res.json({  
      message: "Deleted"  
    });  
  
};
```

studentRoutes.js

```
const express = require("express");  
  
const router = express.Router();  
  
const controller =  
  require("../controllers/studentController");  
  
router.post("/", controller.createStudent);  
  
router.get("/", controller.getStudents);  
  
router.get("/:id",  
  controller.getStudentById);  
  
router.put("/:id",  
  controller.updateStudent);  
  
router.delete("/:id",  
  controller.deleteStudent);  
  
module.exports = router;
```

app.js

```
const express = require("express");  
  
const app = express();
```

```
const studentRoutes =  
require("./routes/studentRoutes");  
  
app.use(express.json());  
  
app.use("/students", studentRoutes);  
  
app.listen(3000, () => {  
  
    console.log(  
        "Server Running On Port 3000"  
    );  
  
});
```